

摘要

随着大语言模型（Large Language Model, LLM）逐步由云端数据中心向个人计算机、边缘设备及嵌入式平台迁移，有限物理内存与较低存储带宽逐渐成为制约端侧大模型部署的重要因素。在资源受限环境下，模型权重占用大量物理内存，自回归推理产生的 KV Cache 又会随上下文长度和并发会话持续增长；与此同时，传统 mmap 与操作系统 Page Cache 主要依赖缺页和页面回收进行被动调度，难以利用 LLM 逐层执行、MoE 稀疏专家激活以及会话生命周期等结构化访存特征，容易产生重复权重读取、频繁 I/O、缓存抖动以及不同工作集之间的内存竞争。

针对上述问题，本项目基于 llama.cpp 设计并实现面向内存受限环境的运行时内存优化系统 Flex-OS。系统采用“加载期规划 + 运行期治理”的两阶段架构：加载阶段由 Memory Planner 根据 cgroup 内存约束、模型结构、权重规模、KV 预留空间及运行时安全余量生成初始资源配置；运行阶段由 Global Memory Governor 持续感知物理内存压力、数据驻留状态、可回收空间和 I/O 行为，动态协调不同工作集之间的资源竞争。

针对不同数据对象的访问特征，Flex-OS 设计了三类差异化内存管理模块。Dense Flex 利用 Dense 模型固定的 Layer 顺序访问规律维护有限层工作集，并结合权重锁定、异步流式加载和自适应预取减少重复权重 I/O；MoE Buffer 以 Expert 为主要管理对象，根据专家工作集、访问预测、压力感知保护和动态预算维护有限专家缓存；KV Controller 将逻辑会话状态与物理驻留状态解耦，以 Block 为物理管理粒度，通过 RELEASE、OFFLOAD 和 Exact Restore 管理 KV 生命周期，在保留有效历史上下文的同时主动释放低价值 KV 物理页，并根据实际恢复需求完成精确恢复。

在此基础上，Global Memory Governor 统一协调 Dense 权重、MoE 专家、KV Cache 与预取任务的内存和 I/O 资源，通过压力门控、资源候选排序、统一预取预算和内存重分配机制，将实际释放的物理内存存在安全边界内重新分配给当前收益较高的工作集，形成“状态观测—资源决策—执行回收—效果反馈—资源再分配”的运行时治理闭环，实现从局部内存优化向跨模块资源协同的扩展。

实验结果表明，Flex-OS 能够在受限内存环境下有效降低 LLM 推理过程中的真实物理内存占用，并提升模型可运行性与吞吐表现。在统一调度框架的 combined_auto 配置下，Dense 模型相对原生 llama.cpp baseline 的峰值 RSS 由 2801.96 MB 降至 **2597.89 MB**，吞吐由 3.09 tok/s 提升至 **4.73 tok/s**，达到基线的约 **1.53** 倍；MoE 模型峰值 RSS 由 2800.92 MB 降至 **1954.75 MB**，降低约 **846 MB**，吞吐由 7.34 tok/s 提升至 **11.46 tok/s**，达到基线的约 **1.56** 倍。同时，KV 管理实验验证了真实物理驻留控制、按需换出与精确恢复机制，跨模块实验进一步验证了 KV 释放所得物理内存经全局确认后重新分配给 MoE 工作集，并能够在后续会话重访时完成 KV 精确恢复，形成完整的物理内存再分配闭环。

综上，Flex-OS 将操作系统虚拟内存机制与 LLM 模型结构及运行时语义相结合，

实现了从操作系统被动调页向模型感知、压力感知和收益感知的主动运行时内存治理转变。系统在不改变 Transformer 计算语义的前提下，对 Dense 权重、MoE 专家和 KV Cache 实施差异化驻留管理与统一资源协调，为有限 DRAM 和存储带宽条件下的大语言模型本地化和边缘化部署提供了一种可实现、可验证的系统级优化方案。

关键词：大语言模型推理；llama.cpp；运行时内存管理；Dense 权重驻留；MoE 专家缓存；KV Cache；全局内存调度；边缘计算

表 1: 赛题目标完成情况

目标编号	完成情况	说明
目标 1	全部完成	完成对 LLM 推理过程的访存分析，揭示模型权重的逐层顺序访问特征与 KV Cache 随上下文增长的线性增长规律，并验证 MoE 模型中专家激活的稀疏性与局部性特征。
目标 2	全部完成	实现针对模型权重与 KV Cache 的运行时时内存管理机制，通过按需加载、动态驱逐与 KV Cache 物理页回收，在保证推理正确性的前提下降低运行时物理内存占用。
目标 3	全部完成	设计并实现异步预取调度器，构建独立后台加载线程池，实现计算与 I/O 的重叠，并设计自适应窗口调优机制。
总计	全部完成	完成 LLM 推理内存优化系统的核心逻辑设计与实现，并完成内存节省和推理性能验证。

目录

1	项目介绍	4
1.1	项目背景及意义	4
1.1.1	大语言模型快速发展，但内存资源成为端侧部署关键瓶颈	4
1.1.2	LLM 推理具有高度结构化访存特征，为主动内存管理提供优化空间	4
1.1.3	KV Cache 动态增长，需要运行时生命周期管理机制	5
1.1.4	从局部优化到 Weight-KV 的全局 I/O 资源调度	5
1.2	项目目标与分析	5
1.3	项目实现内容	7
1.4	项目完成情况	9
1.5	项目开发计划	10
1.6	项目分工	11
2	现有研究调研概况	12
2.1	LLM 推理的访存行为特征	12
2.2	推理框架	13
2.2.1	llama.cpp 推理框架	13
2.2.2	vLLM 推理框架	14
2.3	内存受限场景下的优化方案	15
2.3.1	权重 I/O 优化：模型参数的存储与访问	15
2.3.2	KV Cache I/O 优化：运行时状态管理	15
2.4	边缘端部署的特殊挑战	16
2.5	现有方案的不足与本项目切入点	17
3	系统总体设计	18
3.1	设计目标与总体思路	18
3.2	系统设计原则	19
3.3	系统总体架构	20
3.3.1	推理执行层	20
3.3.2	全局控制与调度层	21
3.3.3	专用内存管理层	21
3.3.4	底层虚拟内存与存储层	22
3.3.5	系统整体运行流程	22
3.4	系统核心组成	22
3.4.1	Dense Flex：基于 Layer 的权重流式管理	23
3.4.2	MoE Buffer：基于 ExpertGroup 的专家缓存管理	24
3.4.3	KV Controller：KV Cache 运行时管理	25

3.4.4	统一调度与执行机制	26
3.5	系统设计总结	27
4	模块设计与实现	27
4.1	加载期与推理期内存规划及后端选择	28
4.1.1	加载期自动后端选择	28
4.1.2	Dense Planner	30
4.1.3	MoE Budget Planner	32
4.1.4	推理期 CPU Hook: Dense 与 MoE 互斥接入	33
4.2	Dense Flex: Layer 级显式权重驻留	34
4.2.1	Dense Ring 的生命周期	34
4.2.2	Request-Wait-Get-Release 闭环	35
4.2.3	Balanced Locking 与 Pin Policy	36
4.3	MoE Buffer: Expert 级显式驻留	37
4.3.1	Warm Working Set Estimator	37
4.3.2	MoE Predictor: CLG、EAM 与 CCT	38
4.3.3	MoE Group-level Eviction	38
4.3.4	MoE Pressure-Aware Protection	39
4.3.5	MoE Clean Reclaim 与 Dynamic Budget	39
4.4	KV Cache 运行时内存管理	39
4.4.1	Block/Cell 生命周期与 Unified Action	40
4.4.2	Physical Budget View 与 RELEASE-first 控制闭环	41
4.4.3	Cost-aware Hot/Cold Claimant Ranking	42
4.4.4	Exact PREFETCH 与 Graph Gate	42
4.4.5	Transfer Group、Read/Restore Pipeline 与 Prefault	43
4.5	Global Memory Governor	45
4.5.1	Pressure Gate	45
4.5.2	Auction Select 与候选排序	46
4.5.3	Async Action Executor	47
4.5.4	Unified Prefetch Budget	47
4.5.5	Clean Reclaim	47
4.5.6	Reallocation Credit	47
4.5.7	Observation Marker	48
5	系统测试	48
5.1	测试环境	48
5.2	Dense 模型权重管理模块测试结果	49
5.2.1	Repack 优化与基础内存降低	50

5.2.2	Dense 模型权重驻留机制测试	51
5.2.3	风险感知的权重锁定	52
5.2.4	不同权重驻留对象的影响	53
5.2.5	权重预取距离与自适应调节	54
5.3	MoE 模型权重管理测试结果	56
5.3.1	MoE 专家工作集边界	56
5.3.2	MoE 专家缓存自动预算	57
5.3.3	压力感知专家调度	58
5.4	KV Cache 管理模块测试结果	61
5.4.1	KV 真实物理驻留控制	61
5.4.2	RELEASE 与 OFFLOAD 的物理收益归因	63
5.4.3	KV 物理驻留与会话恢复代价	64
5.4.4	KV 恢复路径优化	65
5.4.5	F16 KV 表示宽度与恢复数据量	67
5.4.6	多会话生命周期验证	68
5.5	全局自动调度测试结果	69
5.5.1	容量边界与可运行性	69
5.5.2	代表性运行点的内存与吞吐	70
5.5.3	与 LiteRT-LM 的低内存可运行边界对比	71
5.5.4	跨模块物理内存再分配	72
5.6	系统测试结果总结	73
6	当前系统不足与未来方向	74
6.1	运行时资源决策的自适应能力仍需进一步提升	74
6.2	极端内存约束下仍存在内存占用与推理性能的权衡	75
6.3	系统泛化能力与实验验证范围仍有进一步扩展空间	75
7	大语言模型使用说明	75

1 项目介绍

1.1 项目背景及意义

1.1.1 大语言模型快速发展，但内存资源成为端侧部署关键瓶颈

近年来，大语言模型（Large Language Model, LLM）在自然语言理解、代码生成以及智能交互等领域取得了快速发展。随着模型规模不断扩大，LLM 已经从云端数据中心逐渐向个人计算机、边缘服务器以及嵌入式设备等资源受限环境迁移。相比云端服务器，端侧设备通常具有更有限的物理内存容量、更低的存储访问带宽以及更严格的功耗约束，使得大规模模型的高效部署面临巨大挑战。

传统云端部署方式通常依赖大容量 DRAM、高速显存以及分布式计算资源，将模型参数长期驻留在高速存储中。然而，在内存容量有限的端侧设备中，直接加载全部模型参数往往不可行。以当前主流的 Llama 系列模型为例，即使采用低比特量化技术，数十亿参数规模模型的权重仍然可能达到数 GB 至数十 GB。

现有基于内存映射文件（memory map, mmap）的按需加载方案虽然能够突破物理内存容量限制，但其主要依赖操作系统缺页机制进行被动调度，无法感知 LLM 推理过程中的结构化访问模式，容易产生频繁 I/O 等待以及不可控的 page cache 占用。除静态模型权重外，自回归生成过程中用于保存历史上下文信息的键值缓存（Key-Value Cache, KV Cache）会随着输入长度和生成 Token 数量持续增长，进一步加剧运行时内存压力。

因此，在有限物理内存条件下，如何根据 LLM 推理特点主动管理模型权重、KV Cache 以及运行时数据驻留状态，实现低内存占用与高推理性能之间的平衡，成为当前端侧大模型系统优化领域的重要问题。

1.1.2 LLM 推理具有高度结构化访存特征，为主动内存管理提供优化空间

大语言模型推理过程在计算模式上具有显著的结构化特征：在自回归生成时，模型需依次遍历 Transformer Decoder 的所有层，权重访问严格遵循 $Layer_0 \rightarrow Layer_1 \rightarrow \dots \rightarrow Layer_N$ 的固定顺序，这为从操作系统被动分页向应用层主动内存管理转变提供了前提。

对于 Dense 模型，每一 Token 均完整重复该层级序列，使得权重读取具有高度确定性和可预测的时间局部性。这种层级的顺序访问规律，使得系统能够提前预判未来若干步内的权重需求，从而有机会在数据真正被访问之前主动发起预加载或驻留调整，有效隐藏访存延迟并减少缺页中断。

对于 MoE 模型，虽然模型整体参数规模巨大，但每个 Token 仅依据路由器输出的 Top-K 专家索引激活极小比例的参数。这种“全局稀疏、局部密集”的特征，使得权重驻留策略不必以整个模型为单位，而是可以细化到专家粒度：通过跟踪路由历史或结合请求上下文，推理系统能够预估下一 Token 可能激活的专家集合，并据此动态调整高优先级专家权重在快速存储介质中的驻留比例，同时将低概率专家权重提前换出或延迟加

载，从而在有限内存容量下最大化有效权重命中率。

1.1.3 KV Cache 动态增长，需要运行时生命周期管理机制

大语言模型推理阶段，KV Cache 是另一类关键内存资源，其占用量随上下文长度近似线性增长，在长文本生成、多轮对话及高并发场景下极易膨胀，与静态的模型权重形成激烈竞争，持续挤占有限的物理内存空间。

与权重不同，KV Cache 并非预先加载的静态对象，而是推理过程中动态生成的中间状态，其生命周期依附于特定会话、Token 序列及访问频次，具有明显的“按需产生、随用随存”特征，因而无法采用统一的全局加载/卸载策略。这一差异要求系统必须将 KV Cache 的逻辑存在与其物理驻留状态彻底解耦，转而构建运行时感知的生命周期管理机制。

具体而言，该机制需实时监测各会话的 KV 访问热度与时间局部性，并据此实施动态调度，抑制 KV Cache 随上下文增长而无限膨胀的趋势，将物理内存占用稳定在可控阈值内，避免因长上下文累积导致的内存溢出或频繁交换抖动。最终，该机制在有限资源下显著提升长序列推理的稳定性和并发能力，同时为模型权重驻留保留更充裕的内存预算，使整体资源分配更趋均衡。

1.1.4 从局部优化到 Weight-KV 的全局 I/O 资源调度

现有面向大语言模型推理的内存优化工作往往聚焦于单一资源类型，例如仅优化模型权重的按需加载，或仅对 KV Cache 实施压缩与换出。然而，在实际推理系统中，模型权重、KV Cache、预取缓冲及计算中间结果并非独立存在，它们共同竞争有限的物理内存容量与存储带宽，且相互制约。这类非独立的资源竞争关系决定了任何单一维度的局部优化均无法取得系统整体最优，亟需从全局视角重新设计资源协同策略。

针对上述挑战，本文提出一种面向 LLM 推理的运行时内存治理框架。该框架不预设固定资源配额，而是依据当前推理阶段、模型结构特征以及实时内存压力，动态协调各类资源的驻留比例，并通过实时采集各模块的内存占用、访问延迟和命中率，动态调整各级缓存大小及预取深度，避免局部策略产生的冲突。

通过将模型权重、KV Cache 和计算中间结果等数据统一抽象为可管理的资源实体，并通过系统级调度机制实现多目标协同，系统实现了从“数据访问预测”到“资源驻留控制”再到“压力感知回收”的完整闭环，使得大语言模型能够在受限物理内存环境下稳定运行，显著提升长上下文、多会话及混合专家场景下的吞吐与响应性能。

1.2 项目目标与分析

针对大语言模型在资源受限环境下推理时面临的内存容量不足、I/O 延迟显著以及运行时资源竞争等问题，本项目结合 LLM 推理过程固有的结构化访存规律，引入操作系统虚拟内存管理思想，设计面向推理全过程的运行时内存治理框架。该框架旨在突破

物理内存容量限制，在有限资源下实现稳定、高效的推理服务，具体围绕以下四个层面展开。

1. 分析 LLM 推理过程中的访存行为特征

首先基于 llama.cpp 的执行流程，对模型权重、KV Cache 及运行时缓冲区的访问规律进行系统分析：

- 针对 Dense 模型，重点考察 Decoder Layer 顺序执行中权重的访问顺序、层间数据复用关系以及不同驻留策略对 I/O 行为的影响；
- 针对 MoE 模型，聚焦 Router 激活结果导致的专家稀疏访问特征，研究专家权重的访问频率、时间局部性及动态工作集变化规律；
- 对于 KV Cache，分析 KV Cache 随上下文长度增长的内存占用变化，以及多会话场景下不同 KV 状态的生命周期差异。

通过上述分析，将隐式的访存规律转化为可被系统感知和调度的显式信息，为后续主动管理策略提供数据基础。

2. 利用显式驻留管理机制降低运行时物理内存占用

传统 mmap 方案依赖操作系统缺页中断和 page cache 完成数据管理，应用层无法精确控制模型权重及 KV Cache 的实际驻留范围。本项目基于 LLM 结构化访问特点，将权重和 KV Cache 转化为应用层可管理对象，并针对不同数据类型实施差异化的驻留控制策略：

- 面向 Dense 模型，采用 Layer-level 权重流式管理，仅保留当前计算窗口内必要的若干层权重，随执行进度动态切换驻留集合；
- 面向 MoE 模型，实施 Expert-level 工作集管理，根据路由预测和历史访问统计，缓存实际激活或具有高复用价值的专家参数；
- 对于 KV Cache，根据会话活跃程度和访问热度，区分驻留（Resident）、释放（Release）与恢复（Restore）三种状态，按需调整物理占用。

通过上述动态驻留控制，使推理过程能够在远小于完整参数规模的物理内存中完成，有效突破容量瓶颈。

3. 利用预测预取机制隐藏数据加载延迟

在显式限制工作集后，部分数据需要在推理过程中动态加载。若采用同步读取方式，每次缺失均会造成推理线程阻塞。由于 LLM 推理过程具有高度可预测性，本项目设计异步预取机制，实现计算与 I/O 的重叠：

- Dense 模型依据当前 Layer 执行位置，提前预取后续若干层权重，将层级顺序访问的确定性转化为预取窗口；

- MoE 模型根据专家访问历史与 Router 输出预测，提前准备未来可能激活的 Top- K 专家参数，降低稀疏访问带来的加载抖动；
- 在 KV Cache 恢复场景中，利用上下文窗口移动的规律，提前加载即将需要的历史状态，掩盖恢复延迟。

预取操作由独立的后台线程异步执行，与推理流水线并行，从而有效降低存储访问对 Token 生成速度的影响。

4. 构建 Weight-KV-I/O 全局资源协调机制

模型权重、KV Cache 和预取数据共同竞争有限的物理内存资源，单一模块的独立优化难以取得系统整体最优。为此，本项目进一步设计全局资源协调器 (Global Memory Governor)，将 Dense 权重、MoE 专家权重、KV Cache 以及预取缓冲区统一纳入运行时管理体系。该协调器综合考量以下因素：

- 当前物理内存压力和剩余可用容量；
- 各模块当前驻留状态及可回收空间；
- 各类数据的历史访问频次与预期访问收益；
- 存储带宽占用情况及 I/O 竞争程度。

基于上述输入，Global Memory Governor 动态决策各类数据的驻留、回收与预取优先级，并在不同模块间重新分配内存预算。通过全局视角的协同调度，系统从传统局部缓存优化演进为面向 LLM 推理全过程的闭环内存治理框架，在保证服务质量的前提下最大化有限内存资源的利用率。

1.3 项目实施内容

针对上述目标，本项目围绕 LLM 推理过程中的权重管理、KV Cache 管理以及全局资源调度三个方向展开系统设计与实现。整体实现划分为四个主要部分：基础环境与访存特征分析、分层资源管理机制（涵盖 Dense 权重、MoE 专家和 KV Cache）、全局协同调度与资源再分配，以及系统集成与验证。各部分内容如下：

1. 基础环境搭建与访存特征分析

首先在 Ubuntu 22.04/24.04 环境下搭建基于 llama.cpp 的开发与测试平台，完成模型加载流程分析、源码修改及性能测试工具配置。项目使用 GCC、CMake 等工具编译推理框架，并结合 cgroup Global Memory Governor、taskset、perf 及 RSS/内存监控接口构建资源受限实验环境，以模拟端侧设备中的内存约束场景。

在此基础上，基于 llama.cpp 源码与 GGUF 模型文件结构，对模型参数布局、张量组织方式及运行时访问路径进行系统分析。重点考察 Dense 模型的 Layer 顺序

访问规律、MoE 模型的 Expert 稀疏激活特征、mmap/page cache 的驻留行为、权重加载与计算阶段的时序关系，以及 KV Cache 随上下文长度增长的内存占用趋势。通过日志埋点和运行时统计，建立 LLM 推理过程的访存行为模型，为后续主动管理提供数据驱动基础。

2. 分层资源管理机制：权重与 KV Cache 的差异化控制

针对不同数据类型，本项目设计了三套独立但协同的运行时管理模块，分别应对 Dense 权重、MoE 专家权重和 KV Cache 的资源治理需求：

(1) Dense Flex 权重管理模块：针对 Dense 模型所有层权重均需访问的特点，实现层级权重流式管理。核心机制包括按执行进度滑动工作窗口、动态调整窗口大小和平衡锁定等。通过控制层级权重工作窗口大小，系统仅保持当前推理阶段高价值权重驻留，显著降低 Dense 模型运行时的峰值内存。

(2) MoE Buffer 专家工作集管理模块：针对 MoE 模型专家规模庞大但单 Token 激活稀疏的问题，实现专家级缓存管理。系统以单个 Expert 为基本管理粒度，构建专家组 (Expert Group) 工作单元，并设计专家切片流式加载、热点专家保护、以及动态专家预算调整等机制。通过仅缓存高概率访问的专家，将完整专家集合转化为有限大小的动态工作集，在保持较高命中率的同时大幅压缩内存占用。

(3) KV Cache 运行时管理模块：针对长上下文推理中 KV Cache 持续增长的问题，实现运行时生命周期管理。该模块持续监测 KV 块的驻留状态与可回收状态，根据系统内存压力动态执行释放策略、序列级换出以及会话恢复机制，并通过 KV Cache 软预算约束控制整体内存占用上限。通过将逻辑会话状态与物理驻留解耦，低频访问的 KV 数据可及时释放物理内存，为其他模块腾出空间。

3. 全局协同调度与资源再分配闭环。

为避免各模块独立优化导致的资源冲突，本项目设计统一的 Global Memory Governor，统筹协调 Dense 权重、MoE 专家及 KV Cache 之间的内存竞争。Governor 周期性获取系统内存压力、各模块当前驻留状态及可回收空间，并根据优化目标生成对应控制策略，包括数据预取、缓存收缩、资源回收等操作，实现全局资源的最优分配。

在此基础上，进一步设计 Unified Prefetch Budget 机制，防止 Dense、MoE 和 KV 恢复过程同时触发大量预取请求造成瞬时内存尖峰。因此，Governor 根据当前系统可用内存空间、内存压力状态以及存储访问负载，动态调整各模块的预取预算，在隐藏 I/O 延迟和控制额外资源开销之间取得平衡，提升系统稳定性。

此外，项目引入 Reallocation Credit 资源再分配机制，形成“释放—收益—再投资”的闭环。当系统执行权重清理回收、缓存预算收缩或 KV Cache 释放等操作后，系统根据实际释放效果生成内存资源额度，并允许在安全内存范围内将这些额

度重新投资于高收益权重驻留、专家工作集扩大或关键数据预取能力提升，从而实现资源的动态优化循环利用。

4. 系统集成与实验验证

将上述所有模块统一集成至 llama.cpp 推理流程，完成模型加载测试、内存压力测试、RSS 变化分析、KV 长上下文测试、Weight-KV 组合测试以及多模型兼容性测试。通过对比实验，验证系统在有限物理内存条件下能否有效降低运行时 RSS、维持稳定推理吞吐，并避免因内存不足导致的 OOM 或频繁交换抖动。最终评估整体框架在端侧资源受限场景下的实用性与性能增益。

1.4 项目完成情况

在决赛阶段，本项目已经完成面向资源受限环境的大语言模型运行时内存治理系统设计与核心功能实现。系统基于 llama.cpp 推理框架，将模型权重、KV Cache 以及预取数据统一纳入应用层可控的资源管理体系，实现了从单点权重优化向 Weight-KV-I/O 全局协同优化的扩展。

目前系统已经完成以下核心模块：

- Dense Flex：面向 Dense 模型的 Layer-level 权重流式管理；
- MoE Buffer：面向 MoE 模型的 Expert-level 工作集管理；
- KV Controller：面向长上下文场景的 KV 生命周期管理；
- Memory Planner：基于内存约束自动生成初始资源配置；
- Global Memory Governor：面向全局资源竞争的运行时调度框架；
- Unified Prefetch Budget：跨模块预取资源协调机制。

系统已完成端到端集成，并在不同模型规模和内存限制条件下开展测试验证。

表 2: 项目核心功能完成情况

实现内容	对应目标	完成情况	说明
基础环境搭建与访存特征分析	目标 1	完成	完成 llama.cpp 编译部署与 cgroup Global Memory Governor 资源受限环境搭建，建立访存行为分析模型。

实现内容	对应目标	完成情况	说明
加载期 Memory Planner	目标 2、4	完成	根据系统内存大小、模型规模与结构类型, 自动生成初始资源配置, 减少人工调参依赖。
Dense Flex 权重管理模块	目标 2、3	完成	面向 Dense 模型实现 Layer-level 权重流式管理, 有效控制常驻内存集大小。
MoE Buffer 专家管理模块	目标 2、3	完成	以单个专家为粒度构建 Expert-Group 工作单元, 将全局专家集转化为有限动态工作集。
KV Cache 运行时管理模块	目标 1、2	完成	实现 KV 块常驻与可回收状态监测与转换, 通过逻辑与物理驻留解耦降低长上下文内存占用。
统一预取与 I/O 重叠调度	目标 3、4	完成 核心框架	根据系统可用内存空间与 I/O 负载动态分配预取额度, 实现后台异步预取与推理计算并行执行。
全局资源协调与再分配机制	目标 4	完成 核心框架	实现 Global Memory Governor, 周期性采集内存压力及各模块状态, 将释放资源转化为可再投资的信用, 形成释放—收益—再分配闭环。
系统集成与验证	目标 1、2、3、4	完成验证	完成各模块在 llama.cpp 推理流程中的统一集成, 开展 RSS 控制、KV 长上下文、Weight-KV 组合及多模型兼容性测试, 验证有限内存条件下的推理稳定性。

因此, 本项目当前已经完成赛题要求的核心功能, 并进一步形成面向未来端侧大模型部署的全局运行时内存治理框架。

1.5 项目开发计划

表 3: 项目开发计划

阶段内容	时间安排
LLM 推理机制调研、llama.cpp 源码分析、GGUF 文件结构研究以及实验环境搭建	第 1-2 周
完成基础权重流式加载机制，实现 Dense Layer Streaming、MoE Expert Buffer 原型，并完成正确性验证	第 3-5 周
实现 KV Cache 生命周期管理机制，包括 resident 状态分析、释放策略以及长上下文测试	第 5-6 周
实现异步预取机制，完成计算与 I/O 重叠优化，并进行参数调优	第 6-7 周
完成 Memory Planner 与 Global Memory Governor 基础架构，实现 Weight-KV-Prefetch 统一资源管理	第 8 周
完善 Unified Prefetch Budget、Dynamic Budget、Reallocation Credit 等全局优化机制	第 9 周
开展 Dense、MoE、KV 以及 Global Memory Governor 独立消融实验，分析各模块贡献	第 10-11 周
针对不同模型规模、不同 memory cap 和不同存储条件开展稳定性测试	第 12 周
进一步优化 I/O 路径、缓存策略以及资源调度策略，扩展系统对于长上下文、多会话场景的适应能力	第 13-14 周

1.6 项目分工

本项目围绕“模型权重主动管理”和“运行时内存资源治理”两条技术路线展开，并共同完成全局 Global Memory Governor 的系统集成。

表 4: 项目分工

小组成员	分工内容
苏安炫	负责 LLM 权重侧优化相关工作，包括 llama.cpp 框架与 GGUF 模型格式分析，Dense Flex 权重流式管理、MoE Buffer 专家缓存机制、权重预取调度以及相关性能测试。同时负责 Memory Planner 中权重侧预算规划接口设计。

小组成员	分工内容
李思甜	负责 KV Cache 与运行时内存管理相关工作，包括 LLM 推理访存分析、KV Cache 生命周期管理、压力检测、释放恢复机制以及长上下文压力测试。同时负责 Global Memory Governor 中 KV 资源状态接口设计。
共同负责	负责系统总体架构设计、Global Memory Governor 集成、Weight-KV-Prefetch 全局资源协调机制设计、实验环境搭建、Benchmark 测试、结果分析以及参赛文档和答辩材料整理。

2 现有研究调研概况

本节主要介绍和项目有关的大语言模型推理技术、内存管理方案以及边缘部署优化方案的研究情况，并根据调研结果寻找本项目的切入点。

2.1 LLM 推理的访存行为特征

LLM 推理过程具有独特的访存行为特征，这是本项目所有优化设计的出发点。LLM 推理通常分为 Prefill 阶段和 Decode 阶段。

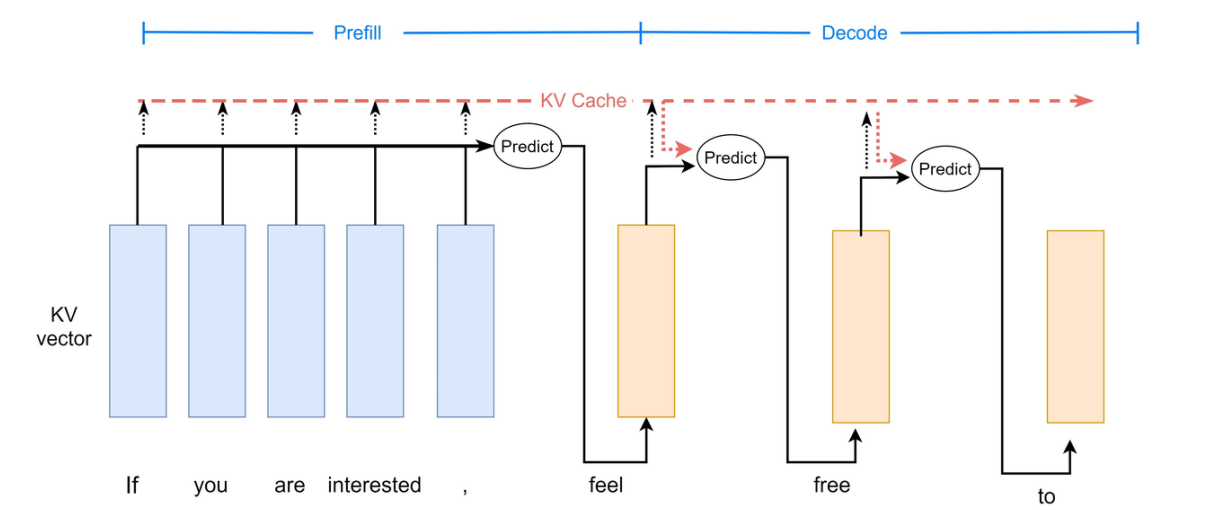


图 1: LLM 推理中的 Prefill 与 Decode 阶段

Prefill 阶段并行处理全部输入 Token，在每个注意力层为每个 Token 计算 Key 和 Value 向量，通常具有较高并行度。Decode 阶段以自回归方式逐 Token 生成，每个新 Token 需要对所有先前 Token 的 Key-Value 对进行注意力计算，因而更容易受内存带宽限制。

在访存行为方面，LLM 推理过程中的数据搬运可分为三个独立的 I/O 流：

1. 权重 I/O

每解码一步需要加载模型权重。对于大规模模型而言，权重 I/O 是推理过程中最主要的数据搬运来源之一。权重 I/O 与批次大小无关，可通过批处理进行摊销。

2. KV Cache I/O

每层每序列需要读取历史 Token 的键值缓存。KV Cache I/O 随序列长度和批次大小线性增长，在长上下文场景下可能接近甚至超过模型权重本身。

3. 激活 I/O

激活 I/O 主要来自中间激活张量在 kernel 启动间的读写。Decode 阶段的激活 I/O 通常远小于权重 I/O 和 KV Cache I/O；Prefill 阶段若不使用 FlashAttention，则可能产生较高的中间读写开销。

上述分析表明，权重 I/O 和 KV Cache I/O 是 Decode 阶段的两大主要瓶颈。两者之间存在动态平衡关系：权重 I/O 可通过批次处理摊销，但 KV Cache I/O 随批次和上下文长度线性增长。这一关系是后续优化方案所面临的核心挑战，也是本项目统一管理权重与 KV Cache 的动机所在。

2.2 推理框架

2.2.1 llama.cpp 推理框架

llama.cpp 是一个纯 C/C++ 实现的大语言模型推理引擎，旨在为广泛的硬件设备提供推理能力，从智能手机等边缘设备到多 GPU 服务器集群均可覆盖。其具有如下特点：

1. 纯 C/C++ 实现，零依赖，无需引入 PyTorch 等深度学习框架。
2. 量化作为一等公民，原生内置多种量化格式。
3. 支持 CPU 与 GPU 混合推理，当模型大于显存时可自动拆分至 CPU 内存和 GPU 显存。

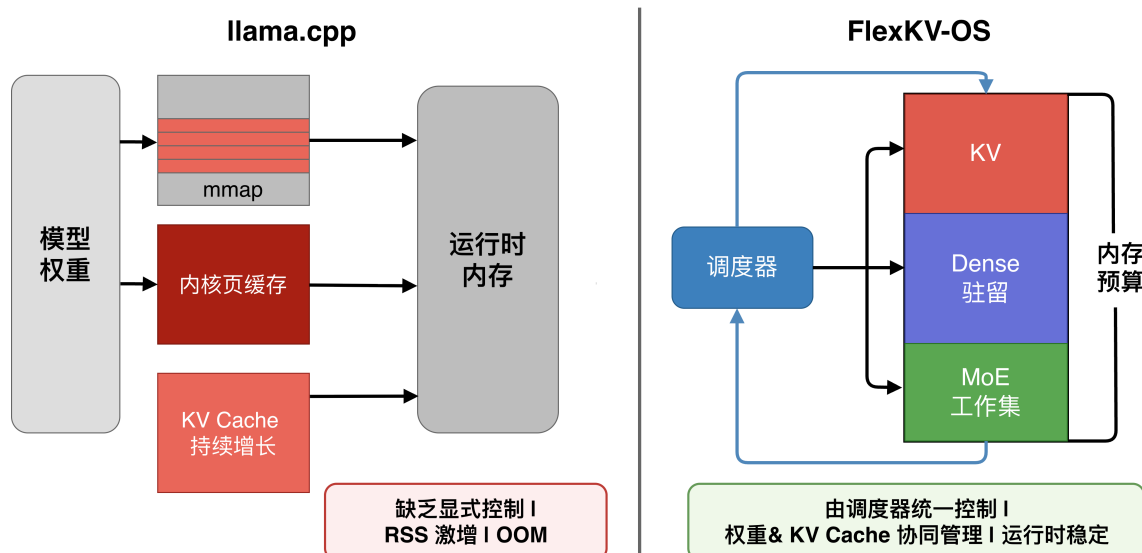


图 2: llama.cpp 推理框架架构

llama.cpp 采用分层架构设计：GGML 层提供底层张量计算、内存管理和后端抽象；llama 核心层实现模型加载、分词、KV Cache 管理、计算图构建及采样功能；工具层提供 CLI、HTTP server 和基准测试工具等。

llama.cpp 当前通过 mmap 机制实现模型参数按需加载。当推理线程访问某层参数时，操作系统通过缺页中断从存储设备加载相应数据页。该机制实现简单，但在物理内存不足时，几乎每次权重访问都会触发 I/O，导致推理吞吐显著下降。本项目使用 llama.cpp 作为基础框架进行二次开发，在其数据加载路径中插入滑动窗口、异步预取、权重流式替换和 KV Cache 回收逻辑。

2.2.2 vLLM 推理框架

vLLM 是一个面向数据中心的高吞吐量 LLM 推理框架，其主要贡献是提出了 PagedAttention 机制。

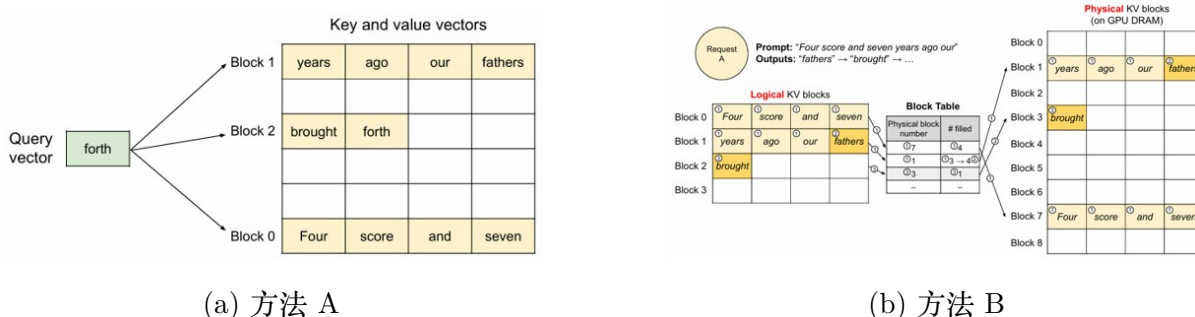


图 3: PagedAttention 机制示意图

PagedAttention 将 KV Cache 切分为固定大小的页或 block，按需分配，其思想与操作系统虚拟内存页表相似。一个 block table 将逻辑页映射到物理内存，使 KV Cache

可非连续存储。PagedAttention 还支持前缀缓存，即多个请求共享同一前缀时，对应的 KV Cache 页可直接复用而非重新计算。vLLM 主要面向 GPU 服务器环境，但其将 KV Cache 视为可灵活管理内存资源的思想，为本项目在边缘端设计 KV Cache 管理策略提供了参考。

2.3 内存受限场景下的优化方案

由前文访存行为分析可知，内存压力主要来源于两类对象：模型权重与运行时 KV 缓存。前者对应参数存储与传输成本，后者对应生成过程中持续增长的内存消耗。因此，内存优化可以从权重 I/O 与 KV cache I/O 两个方向进行探索。

2.3.1 权重 I/O 优化：模型参数的存储与访问

权重开销主要由模型规模决定，其核心问题在于参数无法完全驻留在计算设备中，必须在不同存储层之间迁移。当前主流的方向有：

量化压缩 量化通过降低参数数值精度来减少存储体积，从而降低加载与传输开销。常见做法是将浮点权重映射到低比特整数表示，并在推理阶段进行近似计算。该类方法的特点是压缩比例在部署阶段基本固定，一旦确定量化策略，模型的内存占用也随之固定，因此难以根据实时资源情况进行调整。

分层存储与按需加载 另一类方法通过构建多级存储结构，将权重分布在内存与外部存储介质上，并在计算时按需调入。典型实现包括基于操作系统的惰性加载机制，以及基于用户态控制的主动预取策略。前者依赖访问触发数据加载，行为不可控；后者通过预测未来访问路径提前调度权重，从而减少计算等待时间。

2.3.2 KV Cache I/O 优化：运行时状态管理

KV cache 是自回归推理过程中逐步增长的中间状态，其规模随生成长度线性增加，是动态内存压力的主要来源。现有的优化方案有：

分页式管理 通过将缓存划分为固定大小的块，并使用映射结构管理逻辑与物理地址关系，可以避免连续内存分配带来的碎片问题，并支持多个请求的并发执行。这种方式将缓存管理从连续内存模型转变为块级资源调度问题，使得系统能够在有限内存条件下支持更大规模的并发负载。

缓存回收与迁移 在内存不足或负载变化时，可以将部分缓存数据从高速存储迁移至低速存储，或直接释放不再需要的部分内容。驱逐策略通常基于优先级或重要性估计，例如根据访问频率、位置特征或注意力强度进行判断。

缓存压缩 另一类方法通过降低缓存中数值表示的精度来减少存储占用，但由于缓存状态依赖生成历史，其误差会在后续计算中逐步传播，因此需要在压缩率与质量之间进行权衡。

结构性压缩 从模型结构层面减少缓存规模的方法包括减少注意力头数量、共享键值表示，或使用低维隐变量替代显式缓存表示。这类方法可以从源头降低缓存生成量，而不是在生成后进行压缩处理。

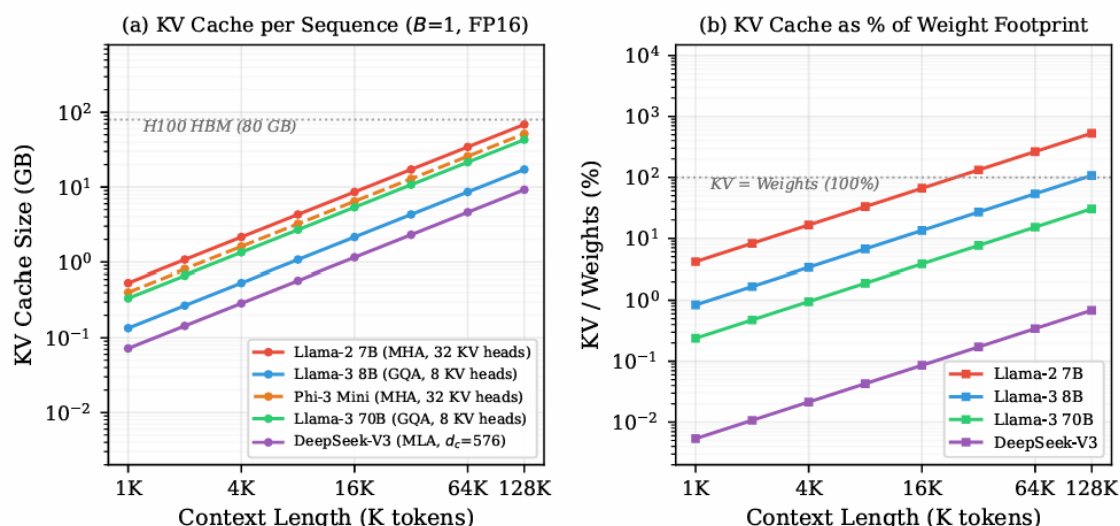


图 4: 架构级 KV 缩减机制

2.4 边缘端部署的特殊挑战

边缘端设备部署 LLM 面临与主流部署环境（如云端、数据中心、服务器集群等）不同的资源约束，其主要挑战体现在以下几个方面：

1. 内存容量严格受限

边缘设备 DRAM 容量通常较小，难以容纳大规模模型权重，即使采用 4-bit 量化仍可能无法完整加载。

2. 存储带宽有限

边缘设备多采用 eMMC 或 UFS 闪存，其顺序读写带宽显著低于主流部署环境中的高性能存储系统，导致模型权重加载与 KV Cache 访问的 I/O 延迟更难被隐藏。

3. 计算能力受限或异构性不足

部分边缘设备仅依赖 CPU 进行推理，或配备的 GPU 计算能力与显存规模有限，难以支撑大规模模型的高效执行。

4. 功耗与散热约束

边缘设备受电池容量与散热条件限制，无法持续维持高功耗计算状态。

上述挑战表明，面向主流部署环境的优化方法难以直接迁移至边缘端，需要针对其存储层次结构与计算资源约束设计专门的系统级优化方案。

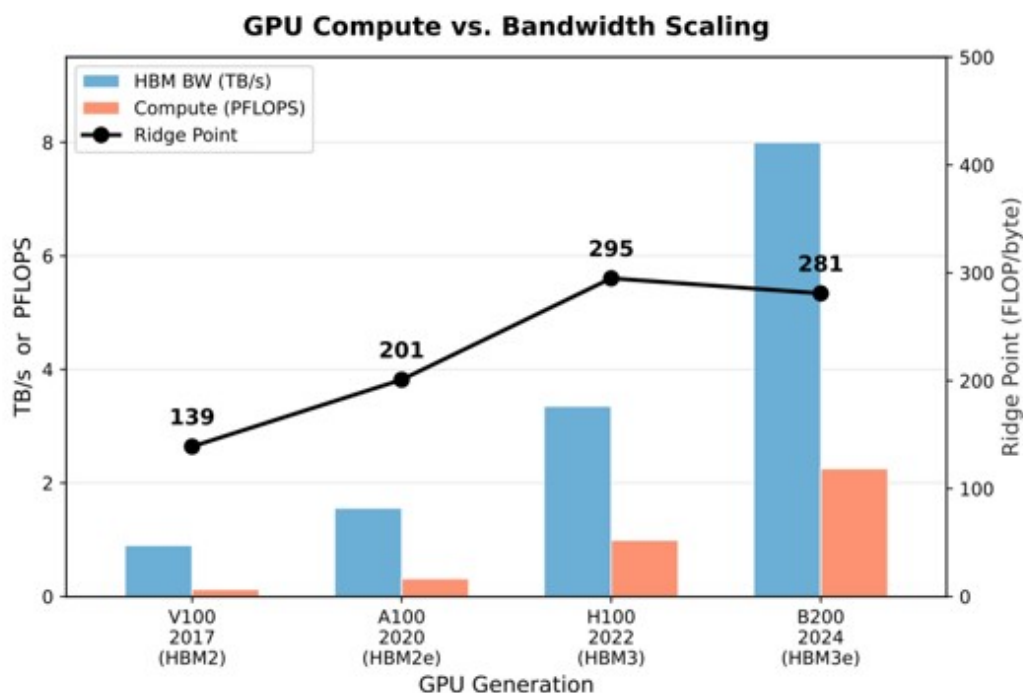


图 5: 边缘端大模型推理资源限制

2.5 现有方案的不足与本项目切入点

综合调研可知，现有方案在内存受限环境下的 LLM 推理中存在以下不足：

1. 权重与 KV Cache 独立管理，未形成统一视图

llama.cpp 通过 mmap 管理权重，通过连续分配管理 KV Cache，两者互不协调。当 KV Cache 增长时，可能挤占权重缓存空间。权重与 KV Cache 在时间维度上存在明确对应关系，但这一关系尚未被充分利用。

2. 权重管理缺乏访存模式感知

现有方案的加载与驱逐策略未充分利用 LLM 推理逐层顺序访问的确定性和可预测性。mmap 方案是被动加载，缺少主动预测；已有预取方案虽具主动性，但未建立显式的层级访问预测模型。

3. KV Cache 管理未适配边缘端 I/O 层次

现有 KV Cache 管理方案多面向 GPU 环境设计，未充分考虑边缘端 DRAM 到 SSD 的 I/O 特性。当 KV Cache 换出至 SSD 时，如何预取、何时换入、如何与权重加载共享 I/O 带宽，仍缺乏系统级方案。

上述问题的核心在于，现有方法通常将权重 I/O 与 KV Cache I/O 作为两个独立优化目标分别处理，而未考虑二者在推理过程中对内存容量与 I/O 带宽的共同竞争关系。这导致单点优化难以从系统层面消除瓶颈。

Configuration	W (GB)	K (GB)	Total (GB)	Dom. Flow	Est. TPOT (ms)
FP16 baseline	141	42	183	W	68
+ INT4 weights (AWQ)	35	42	77	K	29
+ INT4 KV cache (KIVI)	35	10.5	45.5	W	17
+ 2:4 sparsity	17.5	10.5	28	W	10.4
+ Spec. decoding ($\gamma=5, \alpha=0.8$)	4.4*	10.5	14.9	K	5.6

图 6: 权重 I/O 与 KV Cache I/O 孤立优化的瓶颈效应

基于上述分析，本项目构建统一的运行时内存管理框架，将模型权重与 KV Cache 纳入同一调度体系。在该框架下，以层、MoE 专家以及 KV block 为基本管理粒度，对权重驻留状态与 KV Cache 状态进行联合管理；结合 CLG 预测机制实现跨层预取调度，并通过异步加载与驱逐策略实现 I/O 与计算的重叠执行。当权重或 KV block 进入非活跃状态时，系统将其纳入可回收集合，并在资源压力或预测触发下执行换入与换出操作，从而在内存受限环境下实现更稳定的推理执行与更高的资源利用效率。

3 系统总体设计

3.1 设计目标与总体思路

在资源受限环境中，大语言模型推理面临的核心问题并不是单纯降低模型大小，而是在有限物理内存条件下合理决策哪些模型权重需要长期驻留、哪些数据可以按需加载、哪些缓存可以暂时释放、哪些数据值得提前预取，以及回收后的资源应该重新分配给哪个对象。

传统基于 mmap 和操作系统 page cache 的方式能够支持模型运行，但其数据驻留决策主要由操作系统根据缺页访问和页面回收机制完成，无法感知 LLM 推理过程中的高级语义信息——例如当前正在执行的 Transformer Layer、下一阶段可能访问的权重区域、MoE Router 选择的 Expert 集合，以及 KV Cache 当前所属 Session 的生命周期状态。

基于上述分析，本项目的总体设计目标是：利用 LLM 推理过程中的结构化访存特征，在用户态建立面向模型语义的运行时内存治理层，使系统能够主动管理数据驻留状

态，而不是完全依赖操作系统被动调页。围绕这一目标，系统采用“两阶段内存治理”设计：

阶段一：加载期 Memory Planner 在模型加载阶段，系统首先根据当前资源环境和模型结构生成初始内存规划方案。Planner 综合考虑系统可用内存限制、模型权重规模、Dense 或 MoE 模型类型、KV Cache 预留空间以及运行时安全余量，据此选择不同的管理后端：Dense 模型采用 Layer 级权重流式管理，MoE 模型采用 Expert 级工作集管理，KV Cache 根据运行需求建立动态预算，建立适应当前环境的初始资源配置。

阶段二：运行期 Global Memory Governor 由于真实推理过程中的资源需求具有动态变化特点，仅依靠加载阶段规划无法长期保持最优状态，系统进一步引入运行期 Global Memory Governor。Governor 周期性获取系统内存压力、cgroup 内存状态、权重驻留情况、Expert 工作集状态以及 KV Cache 使用情况，并根据当前状态生成回收、释放、扩容、缩容、预取、重映射等控制动作。通过加载期规划与运行期治理相结合，系统能够在保证推理正确性的基础上，根据实际运行状态动态调整内存资源分配。

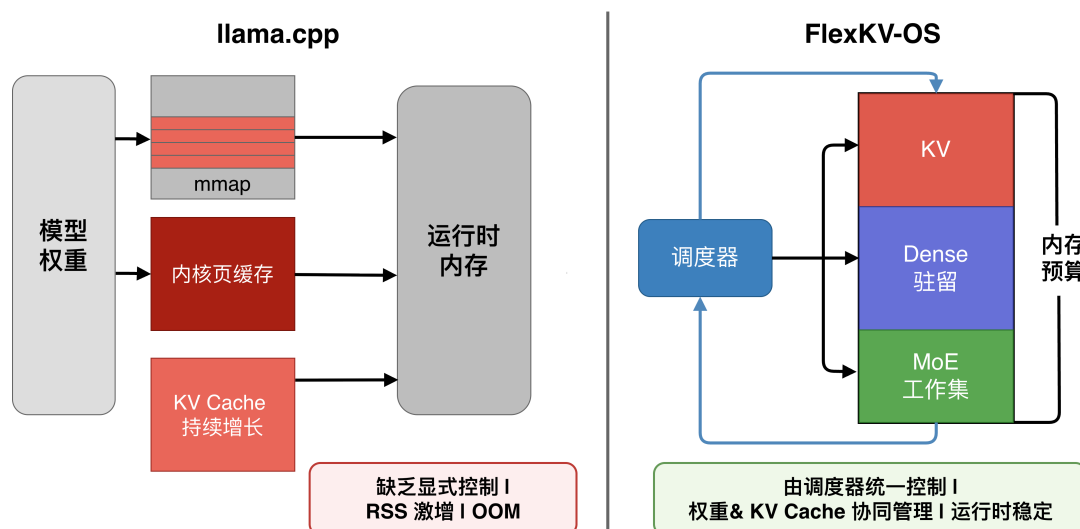


图 7: 系统对比图

3.2 系统设计原则

针对 LLM 推理过程中的访存特点和资源竞争问题，本系统遵循以下设计原则：

1. 模型结构感知的资源管理

不同模型结构具有不同的数据访问模式，因此系统不采用统一缓存策略，而是根据模型结构设计对应的管理机制。通过结构感知设计，使系统能够利用模型自身访问规律提高内存利用效率。

2. 显式驻留优先于被动调页

传统虚拟内存机制主要关注页面访问、缺页处理和页面回收，但无法理解 LLM 推理中的计算语义。因此，本系统将关键数据对象转化为应用层可感知资源，将内存管理从被动调页转变为模型感知的驻留管理，从而减少不可预测的缺页等待。

3. 计算与数据访问解耦

大模型推理性能的重要瓶颈之一是存储访问延迟，系统通过异步加载机制将数据访问过程从推理计算路径中分离，推理线程仅负责发起需求、等待必要数据和执行计算。通过计算与 I/O 重叠，有效降低数据加载等待时间。

4. 全局资源约束优先

由于 Dense 权重、MoE Expert、KV Cache 以及预取缓冲区均共享有限物理内存，局部模块独立优化可能导致资源竞争。因此，本系统引入统一 Global Memory Governor，根据当前全局内存余量、压力状态、I/O 负载与预期收益决定资源调整方向。

5. 回收与资源再利用闭环

传统内存优化通常只关注内存压力触发回收的单向过程，但在多模块 LLM 推理环境中，仅释放资源并不足以获得最佳收益。本系统进一步引入资源反馈机制，将释放资源重新分配给收益更高的数据对象，从而实现内存资源的动态循环利用。

6. 保持推理语义不变

系统所有优化均作用于数据管理路径，而不修改 Transformer 计算逻辑，在降低运行时内存占用的同时，保证模型推理结果的一致性。

3.3 系统总体架构

本系统采用分层式运行时内存治理架构，整体由推理执行层、全局控制与调度层、专用内存管理层以及底层虚拟内存与存储层组成。其中，全局控制与调度层中的 Global Memory Governor 是系统核心控制模块，负责感知运行时资源状态，并协调 Dense 权重、MoE 专家权重以及 KV Cache 三类数据对象之间的资源竞争。系统整体数据流和控制流遵循 *Inference* → *Memory Observation* → *Governor Decision* → *Backend Action* → *Storage/Memory* 的闭环路径，各层职责与协作关系如下：

3.3.1 推理执行层

推理执行层位于系统最上层，负责完成大语言模型前向计算流程，包括 Transformer Layer 计算、Attention 计算、MoE Router 路由、Expert 计算以及 Token 生成等核心任务。该层保持原始推理框架的计算逻辑完全不变，不直接负责数据生命周期管理，而是通过运行时接口向调度层提供计算过程中的状态信息，包括当前执行 Layer、当前 Token

位置、当前激活 Expert 集合、当前 Session 状态以及 KV Cache 访问需求。这些信息作为 Global Memory Governor 进行资源决策的重要依据。在 Dense 模型推理过程中，执行层按照固定 Layer 顺序发起权重访问 ($Layer_0 \rightarrow Layer_1 \rightarrow \dots \rightarrow Layer_N$)；而在 MoE 模型中，则根据 Router 输出动态确定专家访问集合 ($Router \rightarrow Top-K Expert$)。因此，推理执行层能够为上层调度提供模型结构相关的访问信息。

3.3.2 全局控制与调度层

全局控制与调度层是系统核心部分，由 Global Memory Governor、Memory Planner、Prefetch Scheduler 和 Resource Monitor 共同组成，负责将模型访问信息转换为具体资源管理动作，实现从应用层请求到底层内存操作的映射。

Memory Planner 负责加载阶段的初始资源规划。该模块根据模型类型、模型规模、系统内存限制、KV Cache 预留空间以及安全运行余量，生成初始配置，包括 Dense Layer Ring 大小、MoE Expert Buffer 容量、KV Cache 基础预算以及预取初始范围，为后续运行时调度建立合理的资源基线。

Global Memory Governor 是运行阶段动态资源管理的核心。该模块周期性获取系统 RSS、cgroup memory 状态、当前数据驻留情况、可回收空间以及 I/O 压力，并根据当前状态生成对应的控制动作：回收低价值数据、释放暂时不需要资源、扩大高收益缓存、降低资源占用、提前加载数据以及调整长期驻留集合。通过该机制，实现 Dense、MoE 和 KV Cache 三类资源之间的动态协调。

Prefetch Scheduler 根据模型访问规律生成未来数据需求预测。对于 Dense 模型，依据当前执行 Layer 预测后续 Layer 权重需求 ($Current Layer \rightarrow Future Layers$)；对于 MoE 模型，根据 Router 预测结果生成候选专家集合 ($Router Prediction \rightarrow Expert Candidate$)；对于 KV Cache，则根据 Session 恢复状态提前准备需要恢复的 KV Block。所有预取行为均受到统一预算约束，避免过度加载造成额外内存压力。

3.3.3 专用内存管理层

专用内存管理层根据不同数据类型设计不同的管理后端，实现结构感知的数据驻留控制，包含三个核心模块。

Dense Flex 面向 Dense 模型权重管理。由于 Dense 模型具有固定 Layer 顺序访问特点，系统以 Layer 为基本管理粒度，负责 Layer Working Set 管理、权重流式加载、Layer Prefetch、Layer Reclaim 以及驻留集合动态调整。其目标是在保证计算连续性的同时，仅保持当前计算窗口内必要权重驻留。

MoE Buffer 面向专家权重管理。由于 MoE 模型总参数规模巨大但单 Token 仅激活少量专家，系统以 ($layer, expert$) 作为主要管理对象，负责 ExpertGroup 管理、Expert Working Set、Expert Admission、Expert Replacement 以及热点专家保护。通过动态维护有限专家集合，将完整专家空间转化为可控缓存空间。

KV Cache Backend 负责管理推理过程中的动态 KV 状态。系统不直接删除逻辑上的历史 KV，而是区分逻辑 KV 与物理驻留——逻辑上仍属于 Session 的 KV 状态，不一定长期占用物理内存。主要状态包括：RESIDENT（当前驻留内存）、RECLAIMABLE（可安全释放）、SWAPPED（已迁移至外部存储）以及 RESUME_PENDING（等待恢复）。通过状态转换，实现长上下文场景下 KV Cache 的动态生命周期管理。

3.3.4 底层虚拟内存与存储层

底层虚拟内存与存储层由操作系统提供基础资源管理能力，包括 Linux Virtual Memory、mmap 映射机制、Page Cache、DRAM 以及 SSD 或外部存储。系统并不替代操作系统虚拟内存机制，而是在其之上增加模型感知的控制能力，通过 mmap、madvise、O_DIRECT、pread 以及显式 buffer 管理等接口，实现对数据加载、释放和迁移过程的精细控制。

3.3.5 系统整体运行流程

系统运行过程可以概括为以下步骤：推理执行层开始执行当前 Token 计算，并产生 Layer、Expert 或 KV Cache 访问需求；Global Memory Governor 根据当前访问信息和系统状态进行资源分析；对于即将访问的数据，若已驻留则直接执行计算，若未驻留则触发对应 Backend 执行加载操作；Backend 根据调度策略执行权重加载、Expert 加载、KV 恢复或数据回收；完成计算后，系统根据反馈信息更新资源状态，并调整下一阶段资源预算。最终形成 *Compute* $\rightarrow$ *Observe* $\rightarrow$ *Decide* $\rightarrow$ *Execute* $\rightarrow$ *Feedback* 的闭环运行机制。

3.4 系统核心组成

系统核心组成围绕不同类型数据对象的生命周期管理展开，包括模型权重管理模块、KV Cache 管理模块以及统一资源协调机制。由于 Dense 模型、MoE 模型以及 KV Cache 在访问模式和生命周期方面存在明显差异，系统未采用单一缓存策略，而是根据不同数据特征设计对应的管理后端：Dense 模型权重采用基于 Layer 的流式驻留机制，MoE 模型权重采用基于 ExpertGroup 的动态缓存机制，KV Cache 采用基于 Block 的生命周期管理机制。各模块均向 Global Memory Governor 提供资源状态信息，并接受统一的资源调整策略。

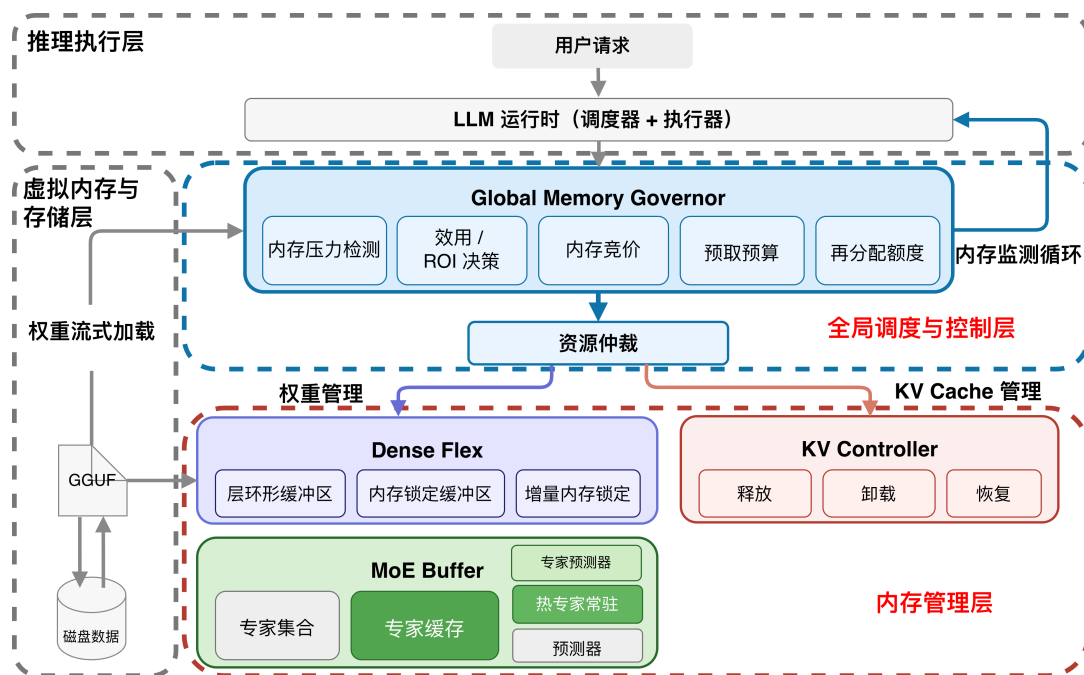


图 8: 主流程图

3.4.1 Dense Flex: 基于 Layer 的权重流式管理

对于 Dense 模型，每个 Token 均需要依次执行全部 Transformer Layer，其权重访问模式为 $Layer_0 \rightarrow Layer_1 \rightarrow \dots \rightarrow Layer_N$ ，具有高度确定性。这意味着系统不需要长期保存全部模型参数，而仅需维护当前计算阶段附近的有效权重集合。Dense Flex 采用 Layer Working Set 管理方式，其核心思想是根据当前执行位置动态维护有限规模的 Layer 驻留窗口，通过提前加载未来访问 Layer 并释放暂时无需求 Layer，降低权重常驻内存。

Layer Working Set 管理根据当前内存预算确定可驻留 Layer 数量，维护当前有效权重集合，主要包括当前正在计算的 Layer、即将访问的预测 Layer 以及根据访问收益长期保留的关键 Layer，其余 Layer 则进入可替换状态。当计算流程移动至新的 Layer 时，系统通过 Layer Streaming 机制根据预测结果提前准备对应权重，数据流路径为 $Storage \rightarrow Weight Buffer \rightarrow Compute$ ，由后台加载线程执行权重迁移，使数据准备过程尽可能与计算重叠。Layer Prefetch 利用 Dense 模型稳定的访问顺序，根据当前 Layer 推测后续访问需求（例如 $Layer_i \Rightarrow Layer_{i+1}, Layer_{i+2}$ ），由后台预取线程提前加载未来 Layer，降低计算线程等待存储访问的概率。当系统检测到内存压力升高时，Layer Reclaim 根据 Layer 最近访问情况、下一次访问距离以及当前驻留收益，选择低收益 Layer 释放，通过动态调整 Layer Working Set，使权重占用能够适应不同内存容量环境。

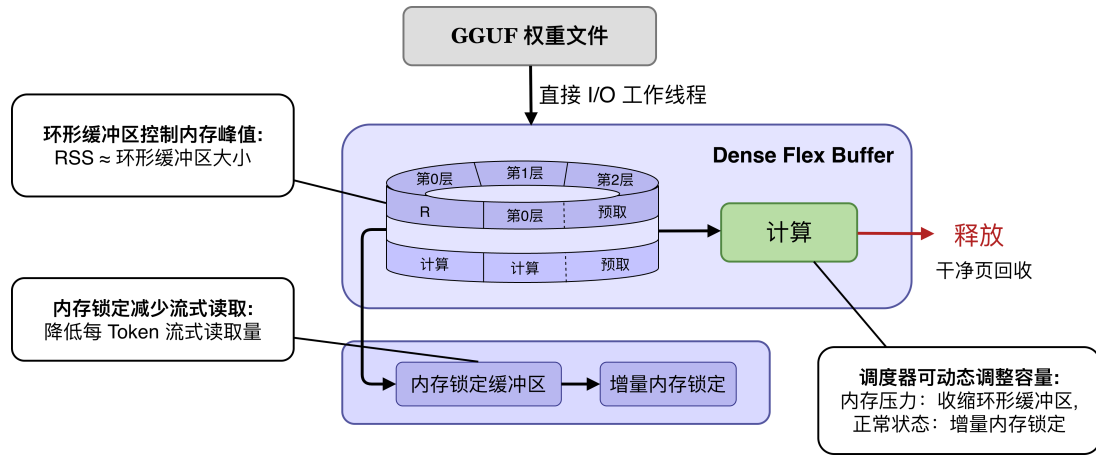


图 9: Dense Flex 模块流程图

3.4.2 MoE Buffer: 基于 ExpertGroup 的专家缓存管理

MoE 模型虽然具有更大的总参数规模，但由于 Router 机制每个 Token 实际仅激活少量 Expert ($Router \rightarrow Top-K \text{ Expert}$)，因此不适合采用 Dense 模型的 Layer 全量管理方式。本系统设计 MoE Buffer 对专家权重进行独立管理。

系统以 $ExpertGroup = (Layer, Expert)$ 作为基本管理单位，相比以整个模型或整个 Layer 为单位管理，该粒度能够精确描述 Expert 激活情况，支持不同 Layer 专家独立替换，降低无效权重驻留。MoE Buffer 不尝试缓存全部 Expert，而是维护有限大小的动态 Expert 工作集，由当前 Token 激活 Expert、预测可能激活 Expert 以及历史高频 Expert 三部分组成，其中高频 Expert 作为热点对象优先保留。

当 Router 产生新的 Expert 需求时，Expert Admission 机制判断当前 Expert 是否已驻留、是否存在可用缓存空间以及是否具有足够访问收益，只有满足条件的 Expert 才进入缓存，避免大量低概率 Expert 造成缓存污染。当 MoE Buffer 达到容量限制时，Expert Replacement 综合考虑 Expert 最近访问时间、使用频率、后续预测概率以及当前热点状态，选择低收益 Expert 执行驱逐。通过上述机制，完整专家集合被转换为有限大小的动态工作集。

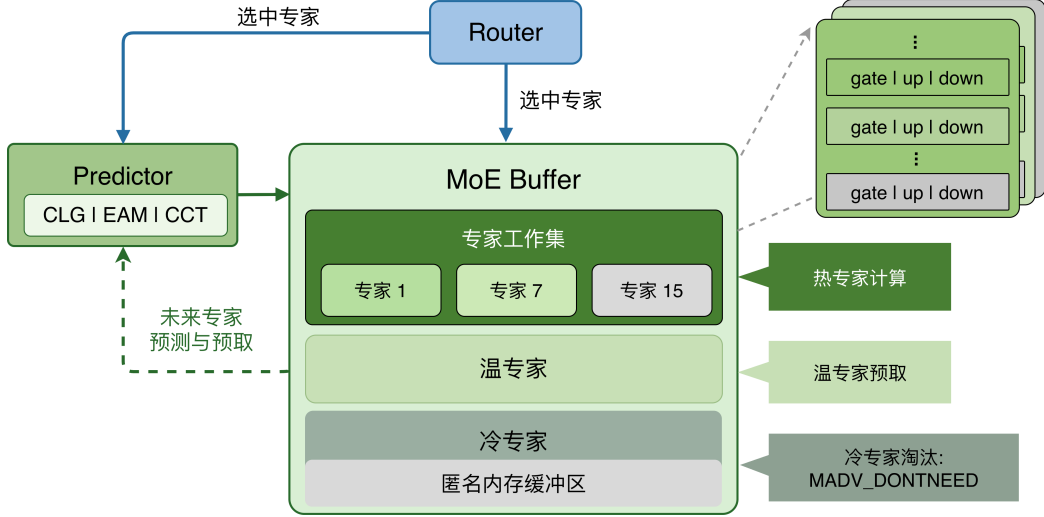


图 10: MoE Buffer 模块流程图

3.4.3 KV Controller: KV Cache 运行时管理

KV Cache 是 LLM 自回归生成过程中的动态运行时状态，其占用量随上下文长度和并发 Session 数量持续增长，无法像模型权重那样静态分配。本系统将逻辑 KV 与物理驻留解耦，即一个 Session 的历史 KV 状态可以继续保持逻辑有效，但不一定始终占用 DRAM，从而在内存受限环境下实现弹性管理。

系统以 Block 为物理管理粒度，并保留原有 Cell 数据布局。每个 Block 记录所属 Session、物理状态以及 active/protected/shared、generation 等运行时信息。根据 KV 生命周期，系统区分 dead/unowned 和 idle-live 两类可回收对象：前者无需后续恢复，直接通过页对齐的 `madvise(MADV_DONTNEED)` 释放物理页；后者仍具有会话价值，通过 OFFLOAD 将完整 KV 数据写入固定后备槽位，写入成功后再释放原物理页并标记为 SWAPPED。所有恢复操作均经过身份与状态校验，恢复未完整完成时禁止后续计算图继续执行，以保证 Attention 计算使用完整、正确的历史 KV。

在预算控制层面，Physical Budget View 提供统一的 resident/reclaimable 物理视图，并由 Physical Resident Budget Controller 根据 soft target 控制 KV 的实际物理驻留。其基本闭环为“采样物理驻留 → 计算 resident excess → 优先 RELEASE dead/unowned → 按需 OFFLOAD idle-live → 重新采样”。其中，Session 重访所需的 correctness-required PREFETCH 优先于 soft resident target，使系统在压缩物理驻留的同时保证必要 KV 能够及时恢复。

对于多个 idle-live Session 之间的 OFFLOAD 对象选择，Cost-aware Hot/Cold 策略根据真实物理收益和历史迁移代价计算候选 Session 的单位物理字节驱逐成本：

$$\text{Score}(s) = \frac{T_{\text{offload}}(s) + P_{\text{reuse}}(s) T_{\text{restore}}(s) + T_{\text{churn}}(s)}{\hat{B}_{\text{exclusive physical}}(s)}.$$

分数越低，表示释放单位真实物理内存所需承担的预期未来代价越小，因此具有更

高的 OFFLOAD 优先级。该评分仅用于已经通过 active、protected、shared 及身份一致性等安全检查的候选 Session；当当前决策缺少可靠的物理驻留或历史成本信息时，统一回退到 idle-age/LRU 排序，避免在同一决策中混用不可比较的评分。Session 恢复后设置短期 resident lease，避免刚恢复的 KV 被立即再次换出。

为验证该生命周期机制在真实多会话条件下能够执行，系统进一步建立 Alibaba usage trace 到真实 llama-server 的多会话回放链路，将逻辑 Session 稳定映射到独立 slot，并按照真实请求到达、空闲、重访和生命周期结束顺序驱动 ACTIVE、IDLE、REVISIT、TTL、DEAD 等状态转换。该链路主要用于验证多会话 KV 生命周期执行的完整性，并为后续不同驻留与驱逐策略的评估提供统一 workload 基础，不作为多会话性能提升结论。

无论采用何种 resident budget 或 OFFLOAD 选择策略，Session 重访时所需的 Exact Restore 与 graph gate 始终作为独立的正确性路径执行，保证性能策略只影响 KV 的物理驻留方式，而不改变模型推理语义。

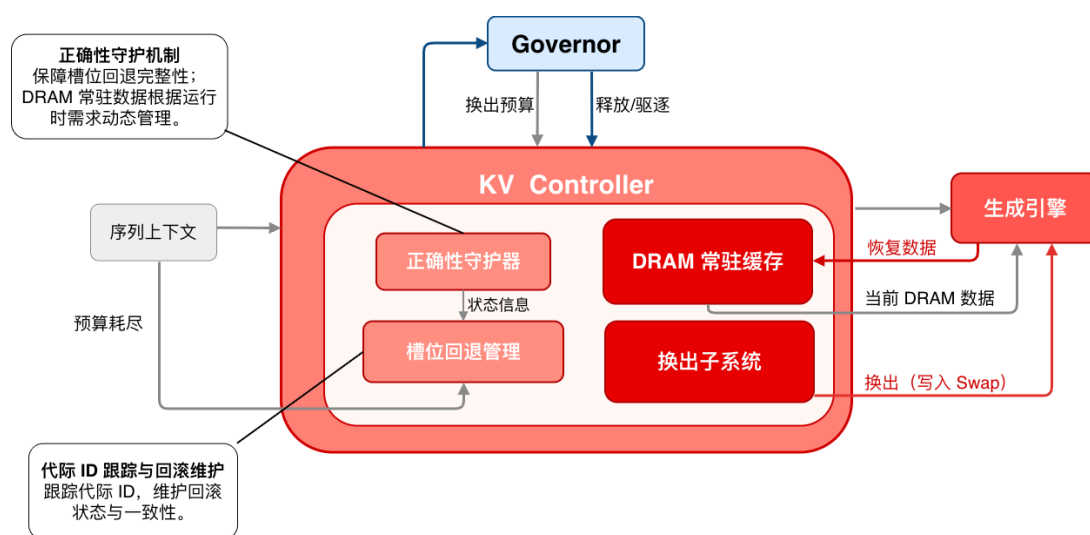


图 11: KV Controller 模块流程图

3.4.4 统一调度与执行机制

由于 Dense 权重、MoE 专家权重以及 KV Cache 均会竞争有限的内存容量和存储带宽，系统需要避免不同模块之间独立决策导致的资源冲突。本项目设计统一的运行时调度机制，由 Global Memory Governor 负责协调不同 Backend 的资源申请、释放和扩展行为。系统整体调度过程抽象为状态观测、资源决策、执行控制和效果反馈四个阶段构成的闭环。

状态观测阶段，系统周期性采集运行时状态，包括当前物理内存占用、cgroup memory pressure、Dense Layer 驻留情况、MoE Expert 工作集状态、KV Block 生命周期状态以及当前 I/O 负载。基于这些观测信息，Governor 在决策阶段判断系统当前处于正常运行状态、内存压力状态、I/O 受限状态或恢复扩展状态，并根据优化目标选择对应策略。执行控制阶段，Governor 向不同 Backend 下发控制动作：Dense Flex 调整 Layer

驻留窗口，MoE Buffer 调整 Expert 工作集规模，KV Backend 执行 Block 回收或恢复，Prefetch Scheduler 调整加载任务。效果反馈阶段，系统根据实际执行结果更新资源状态，包括实际释放内存、缓存命中变化、I/O 延迟变化以及 Token 生成性能变化。通过上述闭环机制，系统从传统的请求驱动加载转变为主动观测、预测、调度、执行与优化的资源治理模式。

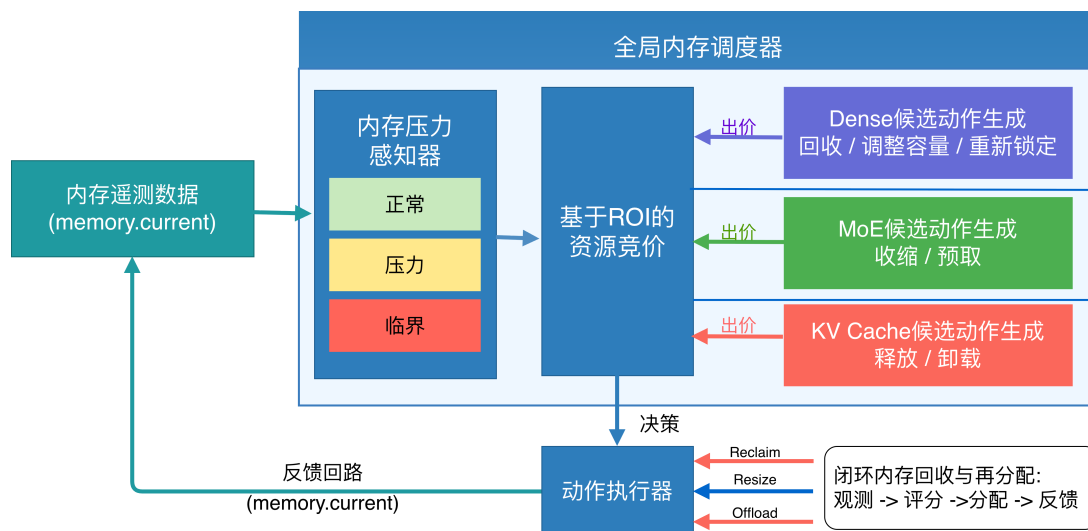


图 12: 统一调度模块流程图

3.5 系统设计总结

综上，本项目针对资源受限环境下 LLM 推理存在的内存容量不足和 I/O 延迟问题，设计了一套面向模型结构感知的运行时内存治理框架，实现了从被动内存管理到面向 LLM 的主动运行时内存治理的转变，为资源受限设备部署大规模语言模型提供了一种系统级优化方案。

4 模块设计与实现

本章从真实源码入口、核心数据结构、关键函数、运行时状态转换和安全边界五个维度说明系统如何落地。本项目不是在 llama.cpp 之外重新实现一套推理引擎，而是在其模型加载、GGML 计算图、CPU Backend、KV Cache 和 Server Scheduler 等既有路径上增加一层结构感知的显式驻留管理与全局内存治理机制。

从代码落点看，系统可以分为四条实现链：

- 加载期规划 (src/llama-model.cpp) 负责 cgroup 感知、模型类型识别、Dense/MoE 后端选择和初始预算规划；
- 权重执行路径 (src/llama-context.cpp、src/llama-flex.cpp、src/llama-moe-buffer.cpp) 实现 Dense Layer Streaming、MoE Expert Streaming 和 Tensor 指针重定向；

- KV 生命周期路径 (KV Memory Backend、tools/server/server-context.cpp) 完成 KV resident/reclaimable 观测、release/offload 及 slot-state fallback;
- 全局运行时治理 (tools/server/server-context.cpp) 涵盖 Pressure State、Candidate、Auction、Unified Prefetch、Async Action 和 Reallocation Credit。

4.1 加载期与推理期内存规划及后端选择

加载期 Memory Planner 负责在模型加载阶段根据 cgroup 约束、模型结构和硬件特性选择最优的显式驻留后端 (Dense Flex 或 MoE Buffer)，并为 Dense 和 MoE 分别制定细粒度的预算、Ring 大小、Ahead 层数和 Lock 策略。推理期则通过 CPU Hook 将后端接入计算图，实现层权重的按需流式加载与释放。整个流程形成“规划 → 后端初始化 → 推理期回调”的闭环，确保在有限内存下平衡性能与安全。

4.1.1 加载期自动后端选择

加载期统一入口位于 src/llama-model.cpp 的 llama_model_base::load_tensors。该函数首先读取 cgroup 状态并统计模型权重规模：

```

1  const auto planner_cgroup = llama_read_cgroup_memory_snapshot();
2
3  const bool memory_governor_auto_by_cgroup = planner_cgroup.valid &&
    planner_cgroup.max_bytes > 0;
4
5  const bool memory_governor_requested = llama_env_flag("
    LLAMA_MEMORY_GOVERNOR") || memory_governor_auto_by_cgroup;
6
7  size_t model_weight_bytes = 0;
8  size_t largest_weight_tensor_bytes = 0;
9
10 for (const auto & it : ml.weights_map) {
11     const size_t nb = ggml_nbytes(it.second.tensor);
12     model_weight_bytes += nb;
13     largest_weight_tensor_bytes =
14         std::max(largest_weight_tensor_bytes, nb);
15 }
16

```

随后，Planner 根据模型 Head 结构估算 KV 每 Token 的字节量：

```

1 size_t governor_kv_bytes_per_token = 0;
2
3 for (uint32_t il = 0; il < hparams.n_layer; ++il) {
4     governor_kv_bytes_per_token +=
5         ((size_t) hparams.n_head_kv(il) *
6         ((size_t) hparams.n_embd_head_k(il) +
7         (size_t) hparams.n_embd_head_v(il)) *
8         sizeof(uint16_t));
9 }
10

```

从系统设计角度，可表示为：

$$B_{KV/token} = \sum_{l=0}^{L-1} n_{head,kv}^{(l)} \left(d_k^{(l)} + d_v^{(l)} \right) b_{elem}$$

Planner 需要同时考虑 Weight Working Set、最大张量/运行时保护空间和 KV Reserve，而非简单地把全部可用 DRAM 都给权重缓存。

模型结构通过 `hparams.n_experts > 0` 判断，在启用 Auto Backend 时，Dense 模型走 Dense Flex，MoE 模型走 Lazy Window + MoE Buffer：

```

1 const bool model_has_moe = hparams.n_expert > 0;
2
3 const bool flex_requested = llama_env_flag("LLAMA_FLEX") || (
4     memory_governor_auto_backends &&!model_has_moe &&!llama_env_is_set("
5     LLAMA_FLEX"));
6
7 const bool use_flex = flex_requested &&!use_mlock &&!ml.check_tensors &&ml
8     .use_mmap &&!params.vocab_only;
9
10 const bool use_lazy_window = !use_flex &&lazy_window_requested &&!
11     use_mlock &&!ml.check_tensors &&ml.use_mmap &&!params.vocab_only;
12

```

其中 `!use_flex` 明确保证 Flex 与 Lazy Window 在同一次 loader pass 中互斥。同时，当 `use_lazy_window || use_flex` 时：

```

1 pimpl->cpu_buf_t_list = make_cpu_buf_t_list(devices, (use_lazy_window ||
2     use_flex)? false: params.use_extra_buf_t, params.no_host);

```

```

3 ml.init_mappings(!(use_lazy_window || use_flex),use_mlock ? &pimpl->
    mlock_mmmaps : nullptr);
4

```

该处理的意义是：当应用层已经建立显式 Weight Residency Backend 时，不再让传统 eager mapping / extra backend buffer 与之重复形成完整权重副本。这一点直接对应后续实验中 repack 导致 RSS 膨胀的问题。

4.1.2 Dense Planner

Dense Planner 不只是简单根据可用内存和层大小计算驻留规模，而是同时考虑非 Layer 固定权重、Runtime Guard、KV Reserve、最大 Layer Size、存储带宽、Compute Proxy、Ring、Ahead、Lock Budget 和风险感知锁定等因素。Planner 遍历所有权重张量，将名称以 blk.N. 开头的张量归入对应 Layer，其余张量（如 embedding、output、final norm）计入 non-layer 固定内存：

```

1 std::vector<size_t> layer_bytes(hparams.n_layer, 0);
2 size_t non_layer = 0;
3 size_t layer_total = 0;
4
5 for (const auto & it : ml.weights_map) {
6     const size_t nb = ggml_nbytes(it.second.tensor);
7     int layer = -1;
8
9     if (std::sscanf(it.first.c_str(),
10 "blk.%d.", &layer) == 1 &&
11 layer >= 0 &&
12 layer < (int) hparams.n_layer) {
13         layer_bytes[layer] += nb;
14         layer_total += nb;
15     } else {
16         non_layer += nb;
17     }
18 }

```

随后根据存储带宽估算 I/O 时间与计算时间的比值 $\rho = T_{IO}/T_{compute}$ ，并据此确定初始 Ahead 和 Ring 大小：

```

1 const double io_ms =bw_mib_s > 0.0? ((double) layer_total /1048576.0) /

```

```

    bw_mib_s * 1000.0: 0.0;
2
3 const double rho =compute_ms > 0.0? io_ms / compute_ms: 0.0;
4
5 int planned_ahead = 1;
6 int planned_ring = 2;
7
8 if (rho < 0.5) {
9     planned_ahead = 3;
10    planned_ring = 5;
11 } else if (rho <= 2.0) {
12     planned_ahead = 2;
13     planned_ring = 4;
14 }

```

在 Lock 预算方面, Planner 为每层 Tensor 建立候选, 估计不同 Lock Budget 下的 Locked Bytes 和 Stream Bytes, 并结合运行时安全余量控制驻留规模:

```

1 const double risk_limit = 0.88;
2
3 const size_t cgroup_cap =(size_t) ((double) planner_cgroup.max_bytes*
    risk_limit);
4
5 const size_t resident_estimate =planner_cgroup.current_bytes +soft_fixed
    +slot * (size_t) planned_ring +lock_budget;
6
7 dense_lock_risk_ratio =planner_cgroup.max_bytes > 0? (double)
    resident_estimate /(double) planner_cgroup.max_bytes: 0.0;

```

同时, 只有当新增 Lock 能够带来足够的 Streaming 收益时才接受:

```

1 const size_t saved =stream > cand_stream? stream - cand_stream: 0;
2
3 const size_t added =risk_lock_cap - lock_budget;
4
5 const bool useful =saved > 0 &&added > 0 &&saved * 4 >= added;

```

通过上述规划, Dense Planner 在有限内存条件下综合确定 Layer 工作集、预取距离和权重锁定规模, 在降低重复权重读取的同时保留必要的运行时安全余量。

4.1.3 MoE Budget Planner

MoE Budget Planner 的规划目标与 Dense 不同：Dense 模型每个 Token 需要依次访问全部 Layer，而 MoE 模型每层仅激活部分 Expert，因此其核心问题是在当前 memory.max 约束下，为主要活跃专家工作集分配可运行的缓存预算。

加载阶段通过张量名称包含 _exps、张量维数为 3 且第三维大于 1 来识别 Expert Tensor：

```
1 const bool is_exps =ggml_n_dims(t) == 3 &&t->ne[2] > 1 &&t.first.find("_exps") != std::string::npos;
```

成功注册至 MoE Buffer 后，该 Tensor 不再交由普通 Window 路径处理。

Planner 首先获取 expert_bytes、warm_working_set、non_expert、KV Reserve 和最大张量等指标：

```
1 const size_t expert_bytes =llama_moe_buffer_expert_bytes(pimpl->
    moe_buffer.get());
2
3 const size_t warm_working_set =llama_moe_buffer_warm_working_set_bytes(
    pimpl->moe_buffer.get(),moe_warm_coverage);
4
5 const size_t non_expert =model_total_bytes > expert_bytes?
    model_total_bytes - expert_bytes: 0;
```

在有限 memory.max 下，Planner 进一步结合安全预算、运行时内存下限和专家工作集需求计算可用预算：

```
1 safe_budget = legacy_safe_budget;
2
3 if (safe_budget < moe_working_set_floor &&hard_safe_budget > safe_budget)
4 {
5     const size_t target =std::min(moe_working_set_floor,
6     hard_safe_budget);
7     safe_budget =std::max(safe_budget, target);
8 }
```

最终 Expert Budget 取 warm_working_set 与 safe_budget 的较小值：

```
1 const size_t budget =budget_unbounded? 0: std::min(warm_working_set,
```

```

    safe_budget);
2
3 llama_moe_buffer_set_budget_plan(pimpl->moe_buffer.get(),budget,
    budget_unbounded,safe_budget,moe_hard_floor);

```

该机制使专家缓存预算同时受到整体运行时内存约束和活跃专家工作集需求的限制。后续实验表明，在 memory.max=1500 MB 的严格内存条件下，固定专家预算均无法稳定运行，而自动预算得到 836 MB 后能够完成 3/3 次测试。

4.1.4 推理期 CPU Hook: Dense 与 MoE 互斥接入

在 src/llama-context.cpp 中,系统通过获取 flex 和 moe 上下文,判断 flex_active 和 moe_active 是否有效,两者互斥:

```

1 auto * flex = model.get_flex_context();
2 auto * moe  = model.get_moe_buffer_context();
3
4 const bool flex_active =backend_cpu != nullptr &&llama_flex_enabled(flex)
5     ;
6 const bool moe_active =backend_cpu != nullptr &&!flex_active &&
7     llama_moe_buffer_enabled(moe);
8
9 if (flex_active) {
10     llama_flex_graph_begin(*flex);
11
12     ggml_cpu_set_weight_stream_callback(llama_flex_stream_callback,
13     flex);
14
15     ggml_cpu_set_op_override_callback(nullptr, nullptr);
16
17 } else if (moe_active) {
18     ggml_cpu_set_weight_stream_callback(
19     llama_moe_buffer_stream_callback,moe);
20
21     ggml_cpu_set_op_override_callback(
22     llama_moe_buffer_mul_mat_id_callback,moe);
23 }
24

```

执行完成后同步并清除回调：

```

1 ggml_backend_sched_synchronize(sched.get());
2
3 ggml_cpu_set_weight_stream_callback(nullptr, nullptr);
4
5 ggml_cpu_set_op_override_callback(nullptr, nullptr);
6

```

该设计确认：Dense 只使用 Weight Stream Callback，MoE 额外允许 Op Override 进入 Sidcar/Fused Compute 路径。

4.2 Dense Flex: Layer 级显式权重驻留

Dense Flex 的核心目标是建立一个具有明确状态的 Layer Residency Backend，通过 Load-time Lock 和 Runtime Delta Pin 降低每 Token 反复读取的数据量。其数据流路径为：GGUF File → Background I/O → Layer Ring → Tensor Pointer Repoint → Compute → Release，形成一个完整的显式驻留管理闭环。

4.2.1 Dense Ring 的生命周期

Ring 作为固定容量的 Slot 池，设总 Slot 数为 N_{slot} ，当前被占用的 Slot 数为 N_{used} ，则 Ring 利用率定义为：

$$\rho = \frac{N_{\text{used}}}{N_{\text{slot}}}. \quad (1)$$

当 ρ 接近 1 时，新请求需驱逐旧层，其 Victim 选择策略依据最近最少使用（LRU）原则，但确保当前 Compute Layer 不被驱逐。

具体实现上，Slot 获取函数（flex_acquire_slot）首先寻找空闲 Slot，若无则选择已释放且状态为 resident 且 last_use 最旧的 Layer 作为 Victim，确保当前 Compute Layer 不会被驱逐：

```

1 for (size_t s = 0; s < ctx.slots.size(); ++s) {
2     if (ctx.slot_layer[s] < 0) {
3         ctx.slot_layer[s] = layer;
4         return (int) s;
5     }
6 }
7 // no free slot
8 for (int l = 0; l < ctx.n_layers; ++l) {

```

```

9      auto & L = ctx.layers[l];
10     if (L.slot >= 0 && L.released &&
11         L.state == layer_state::resident && L.last_use < oldest) {
12         victim = l;
13     }
14 }
15

```

后台 Worker (flex_worker) 循环从队列取任务：获取 Slot、标记 loading、读取未锁定张量、标记 resident，若无可安全 Slot 则 Requeue：

```

1  if (slot < 0) {
2      ctx->stats.queue_requeues++;
3      ctx->queue.push_back(layer);
4      std::this_thread::sleep_for(std::chrono::microseconds(200));
5      continue;
6  }
7

```

数据读取通过 flex_read 实现：普通路径直接使用 pread，O_DIRECT 模式下通过对齐的 Bounce Buffer 完成读取，并分别记录 bytes_streamed 和 bytes_read_phys，从而能够分析 O_DIRECT 对齐导致的真实物理读放大。其物理读放大因子可定义为：

$$\text{ReadAmplification} = \frac{\text{bytes_read_phys}}{\text{bytes_streamed}}. \quad (2)$$

4.2.2 Request-Wait-Get-Release 闭环

预取请求首先检查 Prefetch Budget。系统全局预取预算为 B_{prefetch} ，每个 Layer 的预取消耗等于其未锁定张量的字节数 s 。若 $s > B_{\text{prefetch}}$ ，则丢弃该请求并统计 prefetch_budget_dropped；否则消耗预算并提交。预算更新规则为：

$$B_{\text{prefetch}} \leftarrow B_{\text{prefetch}} - s. \quad (3)$$

Demand 路径将当前 Layer 推入队首并等待完成：

```

1  if (L.state == layer_state::not_resident) {
2      ctx.stats.demand_loads++;
3      ctx.queue.push_front(layer_id);
4      ctx.cv_work.notify_one();
5  }
6  ctx.cv_ready.wait(...)

```

`llama_flex_get_tensor` 的地址优先级为: Locked > Delta Locked > Ring Slot, 三条数据路径最终向 GGML 暴露统一的 `tensor->data` 指针。Release 操作仅将 Layer 标记为 released 并更新 `last_use`, 不立即释放物理页。

4.2.3 Balanced Locking 与 Pin Policy

`llama_flex_finalize` 中保证每层获得相近的 Lock Budget ($\text{lock_bytes}/n_{\text{layers}}$), 避免某些层被完全 Pin 而另一些层每 Token 仍需大量 Streaming。Pin Policy 只决定同一预算内哪些 Tensor 值得常驻:

- 非 Cost-aware 路径: 使用 `flex_sort_for_pin`, 按张量大小降序选择。
- Cost-aware 路径: 使用 `flex_choose_cost_aware_pins` 计算 ROI 选择最优。对于候选张量 t , 其 ROI 定义为:

$$ROI(t) = \frac{\text{Value}(t)}{\text{Cost}(t)}, \quad (4)$$

其中 $\text{Cost}(t)$ 通常为张量字节数, $\text{Value}(t)$ 可基于访问频率或延迟敏感度计算。系统选择 ROI 最高的张量直至预算用尽。

最终统计 `stream_per_token` 作为 Load-time Pin Strategy 的直接机制指标。

Adaptive Ahead 在每次 Layer 边界变化时根据 EWMA 的 I/O、计算和等待时间调整有效 Ahead。设当前 Ahead 层数为 A , 调整量为 ΔA , 则更新规则为:

$$A \leftarrow \text{clamp}(A + \Delta A, A_{\min}, A_{\max}), \quad (5)$$

其中 ΔA 由实测 I/O 延迟与计算延迟之比决定, 且受 Ring Room (空闲 Slot 数) 和 Unified Prefetch Budget 限制。

Clean Reclaim (`llama_flex_reclaim_released`) 对已消费完成的 Layer Slot 执行 `madvise(MADV_DONTNEED)`, 且跳过当前计算层。回收的字节数 R_{clean} 累加到 Governor 的 Raw Relieved 统计中。

Runtime Ring Resize (`llama_flex_resize_ring`) 支持 Grow (分配新 Slot) 和 Shrink (仅删除 Free Slot), 是 Best-effort、Non-destructive 操作。调整后的 Slot 数 N'_{slot} 满足:

$$N'_{\text{slot}} = N_{\text{slot}} + \Delta N, \quad (6)$$

其中 ΔN 可为正或负, 但保证不会释放正在使用的 Slot。

Delta Pin (`llama_flex_delta_pin_async`) 将回收所得内存重新投资于高 ROI Tensor。候选张量 t 的筛选条件为:

- 未锁定 (包括 `delta_locked`)
- 尺寸 > 0
- 字节数 $\leq$ 可用预算
- ROI 不低于阈值 θ (例如 0.8 倍平均 ROI)

候选按 ROI 降序、再按大小、层、名称排序。ROI 计算方式与 Cost-aware Pin 一致。若已有 Delta Pin Job 则拒绝新任务，避免无控制叠加，成功提交后由独立 `delta_pin_worker` 处理 I/O。该机制对应 Governor 的“内存再投资”路径。

综上，Dense Flex 通过 Ring 管理、预算控制、自适应预取和 ROI 驱动的 Pin/Delta Pin，实现了显式、量化的权重驻留管理。

4.3 MoE Buffer: Expert 级显式驻留

MoE Buffer 与 Dense Flex 的核心区别在于管理粒度：Dense 以 Layer 或 Tensor 为基本单位，而 MoE 以 $(layer, expert)$ 即 Expert Slice 为基本管理粒度。MoE Buffer 为完整 $*_exps$ Tensor 建立逻辑上等尺寸的匿名虚拟 Buffer，但只有实际加载过的 Expert Slice 形成物理页驻留，即：

$$\text{Virtual Expert Tensor} \neq \text{Physical Expert Working Set}.$$

这种设计使得系统在逻辑上保留完整的专家参数布局，而物理内存中仅驻留实际被访问的专家切片。

4.3.1 Warm Working Set Estimator

Warm Working Set Estimator (`llama_moe_buffer_warm_working_set_bytes`) 为每个候选 Group 计算：

$$\text{Prior} = \text{seq_access} + \text{access} + \text{hot_score},$$

$$\text{Reuse Distance} = \text{inter_token_gap_ema},$$

$$\text{Gain} \approx \frac{\text{prob} \times \text{group_bytes}}{\text{reuse_distance}}.$$

无历史时退化为 Uniform Prior。随后每层按 Gain 排序，累计到 `warm_coverage` 阈值，从而得到根据 Group Probability、Size 与 Reuse 共同估算的 Resident Set。

4.3.2 MoE Predictor: CLG、EAM 与 CCT

CLG (Cross-Layer Gate): 加载期 Window 与 MoE Buffer 建立连接:

```
1 llama_window_set_moe_buffer(*pimpl->window, pimpl->moe_buffer.get());
2
```

使得 CLG 预测结果直接转为 MoE Buffer 的异步预取请求。

EAM (Expert Access Model): `moe_eam_predict_after_route()` 在 Router 结果后更新转移矩阵，预测未来层 Expert:

$$P(e_{l+d} | e_l).$$

转移概率矩阵 $\mathbf{T}$ 按层维护，其元素 $T_{i,j}^{(l,d)}$ 表示在层 l 访问专家 i 后，在 d 层后访问专家 j 的频率。每次实际路由后，相应计数增加:

$$C_{i,j}^{(l,d)} \leftarrow C_{i,j}^{(l,d)} + 1,$$

然后归一化得到概率:

$$T_{i,j}^{(l,d)} = \frac{C_{i,j}^{(l,d)}}{\sum_k C_{i,k}^{(l,d)}}.$$

预测时，计算每个候选专家的 Score，若 Score 低于阈值（默认 75.0）则跳过:

```
1 if (score < moe_env_f64("LLAMA_LAZY_MOE_EAM_PREDICT_MIN_SCORE", 75.0))
   continue;
2
```

通过阈值的专家进入 Ranked Prefetch Queue。

CCT (Confidence-Controlled Transition): 分支预测风格的饱和置信度，用于 Eviction Protection 和 Retention，而非直接发起预取。

4.3.3 MoE Group-level Eviction

Victim Score 综合考虑多个因子，可抽象为加权线性组合:

$$\text{VictimScore}(g) = \sum_k w_k \cdot f_k(g),$$

其中 f_k 包括: future distance、recent use、hot score、high-sequence protection、active window、reuse、bad reload、ghost reload、CCT、EAM、bit width、speculative-unused state、layer locality 和 pressure。具体加权系数由运行时参数决定。其中 Pressure-Aware Spec Guard 体现为:

```

1 const double pressure_scale = ctx.params.pressure_scale_spec_guard ?
    moe_pressure_scale_locked(ctx) : 0.0;
2
3 const double spec_guard_penalty = speculative_unused &&!spec_evictable
    ? 4.0e8 * (1.0 - pressure_scale) : 0.0;
4

```

当压力升高时，对 speculative-unused 对象的保留惩罚降低，使其更易被驱逐。

4.3.4 MoE Pressure-Aware Protection

Pressure-Aware 保护允许压力动态作用于 pin fraction、layer cap、active window、cooldown 和 speculative guard。其核心为压力比例计算：

$$\text{ratio} = \frac{\text{resident_bytes}}{\text{budget_bytes}}, \quad \text{scale} = \begin{cases} 0, & \text{ratio} \leq \text{soft}, \\ \frac{\text{ratio} - \text{soft}}{\text{hard} - \text{soft}}, & \text{soft} < \text{ratio} < \text{hard}, \\ 1, & \text{ratio} \geq \text{hard}. \end{cases}$$

有效参数按比例缩放，例如：

$$\text{effective_pinned_fraction} = \text{pinned_fraction} \times (1 - \text{scale}),$$

但保留最小下限（floor）。该机制防止保护策略在高压力下锁死 Resident Set。

4.3.5 MoE Clean Reclaim 与 Dynamic Budget

Clean Reclaim: llama_moe_buffer_reclaim_clean 回收 Cold Resident Expert-Group，通过 MADV_DONTNEED 释放物理页。回收量 R_{moe} 累加到全局 Raw Relieved 统计，并计入 Reallocation Credit 的 Pending 环节。

Runtime Budget: llama_moe_buffer_set_budget 允许 Governor 控制下动态 shrink/grow。设当前预算为 B_{moe} ，新预算为 B'_{moe} ，则：

$$B'_{\text{moe}} = \text{clamp}(B_{\text{moe}} + \Delta B, B_{\min}, B_{\max}),$$

其中 ΔB 由 Reallocation Credit 分配的 $\Delta \text{Budget}_{\text{moe}}$ 决定（参见前文积分系统）。Shrink 操作仅释放空闲 Slot，不影响已驻留专家；Grow 操作尝试分配新匿名 Buffer，用于未来加载。

4.4 KV Cache 运行时内存管理

KV Cache 与模型权重的本质区别在于：权重具有稳定的文件后备，而 KV 是随请求运行时生成、并与 Session 历史语义绑定的状态数据。因此，KV 不能仅依赖操作系

统被动回收文件页，而需要同时管理逻辑历史、物理驻留、后备副本、恢复成本与会话生命周期。本系统将 KV 子系统划分为三层：**Exact** 物理生命周期执行层、**Physical Resident Budget** 控制层、**Cost-aware Hot/Cold** 驻留选择层。其中 KV core 负责 block/cell ownership、状态转换、backing I/O、transaction/generation 与真实物理动作，Server/Governor 负责 Session 生命周期、resident budget 与 claimant policy。

4.4.1 Block/Cell 生命周期与 Unified Action

系统以 block 为物理执行粒度、以 cell 为 KV 数据布局粒度，维护 UNUSED/RESIDENT/RELEASED/SWAPPED/PENDING_WRITE/INVALID 等状态。每个 Sequence 通过 seq/epoch 绑定逻辑历史，每次物理动作同时携带 object/generation，用于识别 cache object 重建、mapping 更新或旧事务完成事件。RELEASE 用于销毁 dead/unowned KV 的物理页和逻辑占用；OFFLOAD 用于将仍可能复用的 idle live KV 写入固定槽位 backing 后回收原物理页；PREFETCH 则将 SWAPPED block 恢复到原 KV tensor。

为了避免 Server 直接修改 KV 内部 block 状态，系统将所有 destructive/restore 动作收敛到统一 `execute_action()` 接口。Core 首先建立 capability，再对 write transaction、sequence、protection、ownership 与 block state 做二次校验。OFFLOAD 路径的关键安全检查如下：

```

1  if (!result.capability.can_offload) {
2      result.outcome = llama_kv_action_outcome::unsupported;
3      return result;
4  }
5  if (request.seq_id < 0 ||
6      (size_t) request.seq_id >= seq_to_stream.size()) {
7      result.outcome = llama_kv_action_outcome::rejected;
8      result.reason = llama_kv_action_reason::invalid_sequence;
9      return result;
10 }
11 if ((size_t) request.seq_id < paged_prefetch_protected_seq.size() &&
12     paged_prefetch_protected_seq.test(request.seq_id)) {
13     result.outcome = llama_kv_action_outcome::rejected;
14     result.reason = llama_kv_action_reason::protected_sequence;
15     return result;
16 }
17

```

随后 Core 根据 logical-to-physical mapping 收集目标 block；一旦发现 shared ownership、非法映射或非 RESIDENT/SWAPPED 状态即拒绝动作。实际换出只有在 backing

写入成功后才发布 SWAPPED，并通过页对齐的 `madvise(MADV_DONTNEED)` 释放可安全回收的物理页。这样保证了“先获得可恢复副本，再回收原页”的事务顺序，I/O 失败时不会把未完整落盘的 KV 错误标记为可恢复状态。

4.4.2 Physical Budget View 与 RELEASE-first 控制闭环

KV resident budget 直接约束 KV tensor 的真实物理驻留。Core 通过 `sample_kv_physical_budget_view()` 形成同一 object/generation 下的 coherent snapshot，汇总 resident bytes、dead resident reclaimable bytes、authoritative swapped payload、owned/shared block 数以及 transient staging bound。Resident 与 reclaimable 分别复用现有物理采样和 release-budget authority，保证不同模块采用一致的物理内存统计口径：

```

1  const auto resident_sample = sample_kv_resident();
2  if (resident_sample.available) {
3      result.resident_available = true;
4      result.resident_bytes = resident_sample.resident_bytes;
5  }
6
7  const auto release_budget = sample_kv_release_budget();
8  if (release_budget.valid) {
9      result.reclaimable_available = true;
10     result.dead_resident_reclaimable_bytes =
11         release_budget.reclaimable_resident_bytes;
12 }
13
14 result.valid = true;
15
```

Soft Resident Target 下的预算缺口定义为：

$$\text{Excess}_{KV} = \max(0, B_{\text{resident}} - B_{\text{target}}). \quad (7)$$

当 Excess 大于 0 时，Governor 采用 **RELEASE-first**：优先回收 dead/unowned KV，因为这类数据无需 backing I/O，也不存在后续恢复成本；只有当 RELEASE 返回 no-candidate 且 resident 仍高于 target 时，才在后续 decision 中 arm OFFLOAD，从 idle live claimant 中选择一个受控换出。每次状态改变后都重新读取 Physical Budget View，而不是将理论 action bytes 直接当成已经到账的物理内存收益。

这种分层控制把释放量与被释放者分开：Budget View 决定当前是否存在 resident debt，RELEASE/OFFLOAD executor 决定能安全回收哪些 block，而 claimant policy 仅在多个合法 idle Session 之间决定 victim。

4.4.3 Cost-aware Hot/Cold Claimant Ranking

当多个 idle live claimant 同时存在时，单纯按 LRU 或逻辑 KV 大小驱逐容易出现两类问题：一是驱逐对象虽然逻辑容量大，但实际独占驻留页较少，物理收益有限；二是刚刚恢复或很快会再次访问的 Session 被反复 OFFLOAD/PREFETCH，产生额外写盘、restore 与 page population 开销。因此系统为每个 claimant 构建 decision-scoped feature view，结合当前独占物理可释放量、历史 OFFLOAD 写成本、restore gate、reuse 信号与 round-trip churn 计算单位物理字节的未来迁移代价。

评分逻辑可表示为：

$$\text{Score}(s) = \frac{T_{\text{offload}}(s) + P_{\text{reuse}}(s) T_{\text{restore}}(s) + T_{\text{churn}}(s)}{\hat{B}_{\text{exclusive}}(s)}, \quad (8)$$

其中分母 $\hat{B}_{\text{exclusive}}$ 来自 claimant 级 mincore/ownership 聚合，不使用 logical KV size 冒充 physical relief。对应源码中的核心计算等价于：

```

1 const uint64_t expected_cost_us =
2 history.last_offload_time_us +
3 reuse_probability_ppm * history.last_restore_gate_us / 1000000ULL +
4 params.churn_penalty_us * history.round_trip_count;
5
6 score.total = expected_cost_us * 1000000ULL /
7 physical.estimated_exclusive_resident_bytes;
8

```

Score 越低，表示“释放 1 Byte 真实物理内存所付出的未来代价”越低，因此优先被驱逐。安全 eligibility 始终先于排序：active、protected、shared、epoch/generation 不一致以及处于 resident lease 的 claimant 不参与 destructive action。若任一 eligible claimant 缺少 authoritative physical estimate、write cost、restore cost 或 reuse evidence，则整个 decision 统一采用 idle-age/LRU authority，避免将不同量纲的 score 混合排序。

为了抑制 OFFLOAD-restore-OFFLOAD 的往返抖动，系统在成功 restore 后设置 resident_lease_until_sample，lease 内禁止普通 OFFLOAD；每次真实 OFFLOAD 后再 restore 会增加 round_trip_count，并通过 churn penalty 提高下一次驱逐成本。普通 turn rollover 迁移 claimant history，只有 prompt clear、object/generation 变化等真实 lineage loss 才清空历史，从而使成本反馈能够跨多轮会话持续生效。

4.4.4 Exact PREFETCH 与 Graph Gate

Session 重访时，恢复动作不依赖 Governor 的 soft victim policy，而是进行独立的 correctness-required PREFETCH。Server 在 n_past 确定之后、batch/graph 构建之前

保护目标 sequence，并提交 all-required restore；只有 decision identity、I/O、fail-stop、context 与 shortfall 全部满足要求时，才允许后续 llama_decode()：

```

1 ops.set_protected(seq_id, true);
2 llama_kv_action_request request;
3 request.action = llama_kv_action::prefetch;
4 request.decision_id = decision_id;
5 request.seq_id = seq_id;
6 request.max_blocks = 0;
7 request.correctness_required = true;
8 request.all_required = true;
9 result.action = ops.execute(request);
10
11 result.graph_allowed =
12 result.action.decision_id == decision_id &&
13 (result.action.outcome == llama_kv_action_outcome::completed ||
14 result.action.outcome == llama_kv_action_outcome::no_op) &&
15 !result.action.io_failure &&
16 !result.action.fail_stop &&
17 !result.action.capability.context_invalid &&
18 result.action.shortfall_bytes == 0;
19

```

这条路径保证 policy prediction 只影响性能，不影响正确性：即使前面的 Hot/Cold 判断没有提前保留某个 Session，真实请求到达后仍由 Exact gate 恢复所有 required SWAPPED blocks；partial failure、I/O error 或 shortfall 均会阻断 graph，而不是带着不完整 KV 继续推理。

4.4.5 Transfer Group、Read/Restore Pipeline 与 Prefault

为了降低 Exact restore 的 I/O 与 page population 开销，系统不再按单 block 独立完成完整 read-scatter-commit，而是先将连续 SWAPPED block 合并为 bounded transfer group。Group 内记录 seq/object/generation/mapping generation、begin/end block、cell range 与 total bytes；只有连续 block 且累计字节不超过 group cap 时才合并：

```

1 const bool start_group = tasks.empty() ||
2 block != tasks.back().end_block ||
3 tasks.back().total_bytes > byte_cap ||
4 block_bytes > byte_cap - tasks.back().total_bytes;

```

```

5
6 if (start_group) {
7     paged_restore_group_task task;
8     task.seq_id = seq_id;
9     task.object_id = paged_resident_object_id;
10    task.generation = paged_resident_generation;
11    task.mapping_generation = paged_mapping_generation;
12    task.begin_block = block;
13    tasks.push_back(std::move(task));
14 }
15

```

在此基础上, restore 采用有界的两级流水: 当前 group 完成 read 后, 后台 read worker 提前读取下一 group 到 task-owned staging; scheduler owner 同时对当前 group 执行 identity revalidate、destination prefault、tensor scatter 与 RESIDENT commit。核心调度关系为:

```

1 paged_restore_group_start_async(current);
2 paged_restore_group_wait_async(*current);
3
4 for (size_t i = 0; i < restore_tasks.size(); ++i) {
5     if (i + 1 < restore_tasks.size()) {
6         paged_restore_group_prepare(*next);
7         paged_restore_group_start_async(next); // READ(N+1)
8     }
9
10    paged_restore_group_post_read(*current); // RESTORE(N)
11    paged_restore_group_complete(*current);
12    std::swap(current, next);
13 }
14

```

Read worker 只负责 backing read 与 task-owned staging, 不直接修改 live KV tensor; scatter、状态发布、transaction 与 graph gate 始终由 scheduler owner 完成。这样既能形成 READ(N+1)||RESTORE(N) 的 I/O/恢复重叠, 又避免异步 worker 与 active graph 对同一 KV tensor 并发写入。目标页 prefault 在 scatter 前主动建立目的页, 进一步减少 restore 期间的同步缺页抖动。

综上, KV 运行时形成统一闭环: **Physical Budget View** 判断需要释放多少内

存, RELEASE-first 区分不可恢复与可恢复回收, Cost-aware Hot/Cold 决定优先驱逐哪个 Session, Unified Action 在 core 内执行 block 级安全状态转换, Exact PREFETCH 与 Transfer Group Pipeline 负责恢复, 并由真实 physical relief、restore cost、lease/churn 反馈下一轮决策。这一设计使容量控制、victim selection、I/O 恢复与 correctness gate 共享同一套 block/cell 生命周期和物理身份。

4.5 Global Memory Governor

Global Memory Governor 的输入信号包括: cgroup 内存状态 (memory.current / memory.max / memory.high)、RSS 常驻内存、KV Cache 压力、Dense 模型 Flex 统计、MoE 缓冲区统计、KV 预算、分配动作历史、冷却状态以及重分配信用额度, 输出 Dense/MoE 的回收、预算及预取策略, KV 释放/卸载, 重分配与观测标记动作。

Candidate 结构体统一携带 kind、action、id、score、roi、bytes、reason 等字段, 使 Dense Layer、MoE ExpertGroup、KV Global 和 KV Sequence 进入同一决策接口:

```

1 struct memory_governor_candidate {
2     const char * kind = "none";
3     const char * action = "none";
4     int32_t id = -1;
5     int64_t score = std::numeric_limits<int64_t>::min();
6     double roi = -std::numeric_limits<double>::infinity();
7     uint64_t bytes = 0;
8     const char * reason = "none";
9 };
10

```

4.5.1 Pressure Gate

Pressure Gate 根据运行时内存压力状态限制不同类型的资源动作。压力状态首先由 KV Pressure Sampler 统一采样, 并由 Global Memory Governor 在调度时进一步结合当前工作集和安全余量形成有效压力状态。

当 cgroup memory.max 为有限值, 且未显式配置 RSS 或 cgroup 绝对压力阈值时, 系统默认采用 cgroup 内存占用比例作为压力源:

$$R = \frac{M_{\text{cur}}}{M_{\text{max}}}, \quad (9)$$

其中 M_{cur} 为当前 cgroup 内存占用, M_{max} 为 cgroup 硬内存上限。memory.high 作为运行时观测信息保留, 但不直接参与该压力状态计算。

默认情况下, 系统设置三个压力水位:

$$R_{\text{low}} = 0.70, \quad R_{\text{pressure}} = 0.80, \quad R_{\text{critical}} = 0.95. \quad (10)$$

系统采用 NORMAL、PRESSURE、CRITICAL 和 RECOVERY 四态状态机。内存占用达到 Critical 阈值时立即进入 CRITICAL；进入普通 PRESSURE 状态需要满足连续采样滞回和冷却条件。从 PRESSURE 或 CRITICAL 退出时，则需要内存占用持续低于 Low Water，先进入 RECOVERY，满足恢复条件后再返回 NORMAL。该设计避免压力阈值附近的频繁状态抖动。

当显式配置 RSS 或 cgroup current 的绝对压力阈值时，系统使用对应绝对值作为压力源；若 memory.max 为无限且没有配置有效的绝对阈值，则仅保留压力遥测，不执行状态转换。PSI 信号只用于持续压力下从 PRESSURE 向 CRITICAL 的辅助升级，不会单独触发破坏性回收动作。

Global Memory Governor 在基础压力状态上进一步结合运行时安全余量计算 effective_pressure 并以该状态执行 Pressure Gate。候选动作的门控规则可表示为：

$$\text{Allowed}(c, S) = \begin{cases} \text{Reclaim}(c), & S = \text{CRITICAL}, \\ \text{Reclaim}(c) \vee \text{ResumePrefetch}(c), & S = \text{PRESSURE}, \\ \text{true}, & S \in \{\text{NORMAL}, \text{RECOVERY}\}. \end{cases} \quad (11)$$

其中， $\text{Reclaim}(c)$ 包括 Dense/MoE 的 clean reclaim、KV RELEASE 以及 KV OFFLOAD； $\text{ResumePrefetch}(c)$ 仅表示受到恢复正确性保护的 KV PREFETCH。因此，在高压状态下系统优先执行内存回收，而 Session 恢复所必需的 KV PREFETCH 仍可在 PRESSURE 状态下通过门控，避免内存优化破坏推理正确性。

4.5.2 Auction Select 与候选排序

通过门控的候选将按综合效用降序排列，并累计字节数直到达到 warm_coverage 阈值。

定义候选的回报率 (ROI) 为：

$$\text{ROI}(c) = \frac{\text{score}(c)}{\text{bytes}(c) + \epsilon}, \quad (12)$$

其中 ϵ 为极小正数防止除零。综合排序分采用调和形式：

$$U(c) = \alpha \cdot \text{score}(c) + \beta \cdot \text{ROI}(c) \cdot \text{bytes}(c), \quad (13)$$

默认取 $\alpha = 0, \beta = 1$ 即纯 ROI 排序。

4.5.3 Async Action Executor

异步执行器覆盖 `dense_clean_reclaim`、`moe_clean_reclaim`、`k_v_global_release`、`k_v_sequence_offload` 和 `k_v_sequence_slot_state_offload`，通过原子 Inflight Flag 防止同类动作重复并发：

```

1 if (!inflight.compare_exchange_strong(...)) {
2     memory_governor_async_rejected++;
3     return false;
4 }
5

```

4.5.4 Unified Prefetch Budget

系统维护全局 Tick Budget B_{total} ，按优先次序分配。KV Resume 具有最高优先权，通过 `memory_governor_prefetch_budget_reserve` 预留：

$$B_{kv_reserve} = \min(\text{Demand}_{kv}, B_{total} \times \gamma), \quad (14)$$

其中 γ 为 KV 预留比例上限（例如 0.5）。剩余预算为 $B_{remain} = B_{total} - B_{kv_reserve}$ 。

剩余预算按动态比例 λ 分配给 Flex (Dense) 和 MoE Buffer：

$$B_{flex} = \lambda \cdot B_{remain}, \quad (15)$$

$$B_{moe} = (1 - \lambda) \cdot B_{remain}, \quad (16)$$

其中 λ 根据当前失效率动态调整：

$$\lambda = \frac{\text{FlexMissRate}}{\text{FlexMissRate} + \text{MoEMissRate}}. \quad (17)$$

4.5.5 Clean Reclaim

统一调用包括 Dense 的 `llama_flex_reclaim_released`、MoE 的 `llama_moe_buffer_reclaim_clean` 和 KV 的 `execute_action(release/offload)`。KV 不纳入 `clean_reclaim` API，因其需要保持运行状态语义。

4.5.6 Reallocation Credit

系统将本轮策略调整后获得的内存释放量作为分配来源。该值首先进入挂起状态，随后通过当前内存使用确认环节，结合 `cgroup` 实时内存占用 `memory.current` 进行核实，确保释放量真实反映在空闲内存中。通过确认的部分转化为可分配内存。

为防止内存累积过快或长期空闲，系统引入指数衰减，随时间降低旧空闲内存价值，并辅以保护机制防止过度再分配引发 OOM。最终，经过以上处理的空闲内存被用于再分配。具体量化如下：

设第 t 步的原始释放量为 $R_{\text{raw}}(t)$ ，但物理内存变化需通过 `memory.current` 确认：

$$\text{Pending}(t) = \max(0, -\Delta M_{\text{cur}}(t)), \quad (18)$$

其中 $\Delta M_{\text{cur}}(t) = M_{\text{cur}}(t) - M_{\text{cur}}(t-1)$ 为内存变化（下降为负）。

实际空闲内存采用指数衰减：

$$\text{Earned}(t) = \text{Earned}(t-1) \cdot (1 - \delta) + \text{Pending}(t), \quad (19)$$

$\delta \in [0.05, 0.2]$ 为衰减率。

为防止过度分配，设置上限和守护条件：

$$C(t) = \min(\max(\text{Earned}(t), 0), \text{Cap}), \quad (20)$$

并仅当压力 $P < 0.3$ 时允许分配：

$$\text{InvestAllowed} = \mathbf{1}_{P < 0.3}. \quad (21)$$

可分配的内存 $C(t)$ 按固定比例 $\mu_{\text{moe}}, \mu_{\text{dense}}$ 分流：

$$\Delta \text{Budget}_{\text{moe}} = \mu_{\text{moe}} \cdot C(t), \quad (22)$$

$$\Delta \text{Budget}_{\text{dense}} = \mu_{\text{dense}} \cdot C(t), \quad (23)$$

$$\Delta \text{Budget}_{\text{prefetch}} = (1 - \mu_{\text{moe}} - \mu_{\text{dense}}) \cdot C(t). \quad (24)$$

4.5.7 Observation Marker

输出分为 Planner、Pressure、Dense、MoE、KV、Global 六类关键指标，其中：

- Pressure：压力 P 及其滑动平均 $\text{EMA}(P) = 0.9 \cdot \text{EMA}_{\text{prev}} + 0.1 \cdot P$
- Dense / MoE：当前 Resident Set 覆盖率 $\text{Coverage} = \frac{\sum \text{bytes}_{\text{selected}}}{\sum \text{bytes}_{\text{total}}}$
- KV：Offload 比例 $\text{OffloadRatio} = \frac{\text{Bytes}_{\text{offloaded}}}{\text{Bytes}_{\text{total_kv}}}$
- Global：内存松弛度 $\text{Slack} = \frac{M_{\text{high}} - M_{\text{cur}}}{M_{\text{high}}}$

这些观测为系统自适应调优提供运行期间数据。

5 系统测试

5.1 测试环境

本项目所有实验均在统一物理环境下完成，以保证不同优化模块之间的可比性。测试过程中分别启用或关闭权重管理、MoE Buffer、KV Cache 管理以及调度优化模块，通过环境变量控制运行模式，无需切换推理框架。

测试环境参数如表 5 所示。

表 5: 测试环境参数

环境指标	环境参数
Linux 发行版	Ubuntu 22.04
Linux 内核	5.15.0+
机器模式	物理机
内存大小	23GB
处理器	x86-64, 8 核
存储设备	SSD, 顺序读速约 279MB/s
Dense 测试模型	Llama-3-8B-Instruct-Q4_K_M.gguf
Dense 模型大小	约 4.58GB
MoE 测试模型	Qwen1.5-MoE-A2.7B-20-experts-SFT-trained.Q4_K_M.gguf
MoE 模型结构	24 个 MoE layer, 每层 20 个 expert, top-4 激活
内存限制方式	cgroup Global Memory Governor (memory.max)
测试工具	llama-bench、perplexity、trace replay 工具

本项目测试主要围绕三个目标展开：

1. 验证系统是否能够降低大模型推理过程中的物理内存占用；
2. 验证权重流式加载、缓存驻留以及运行时调度机制是否能够降低 I/O 等待；
3. 验证优化路径是否保持与原始 mmap 推理路径一致的语义。

5.2 Dense 模型权重管理模块测试结果

Dense 权重管理主要针对内存受限条件下权重重复驻留和频繁磁盘读取问题，通过消除冗余副本、维护有限层工作集、风险感知权重锁定以及动态预取等机制控制运行时权重驻留。实验分别验证基础内存开销、权重驻留效果、内存安全边界，以及驻留对象选择和预取距离对推理性能的影响。

实验主要从以下三个方面展开：

1. 权重副本消除带来的基础 RSS 降低；
2. Dense 模型在不同驻留策略下的权重流式性能；
3. 低内存环境下权重预算选择与 I/O 调度能力。

5.2.1 Repack 优化与基础内存降低

首先测试原生权重路径中 Repack 对物理内存占用的影响。llama.cpp 为适配计算布局会生成额外的临时权重副本；在 Flex-OS 接管权重驻留后，这部分中间副本可以直接去除。

实验结果如表 6 和图 13 所示。

表 6: 关闭 Repack 前后的内存占用与推理吞吐

配置	RSS (GB)	吞吐 (tok/s)	说明
Dense 原生路径 (含 Repack)	7.77	8.15	含 Repack 临时权重副本
Dense 关闭 Repack	4.56	8.60	去除冗余权重副本
MoE 原生路径 (含 Repack)	5.73	19.26	含 Repack 临时权重副本
MoE 关闭 Repack	3.60	19.58	去除冗余权重副本

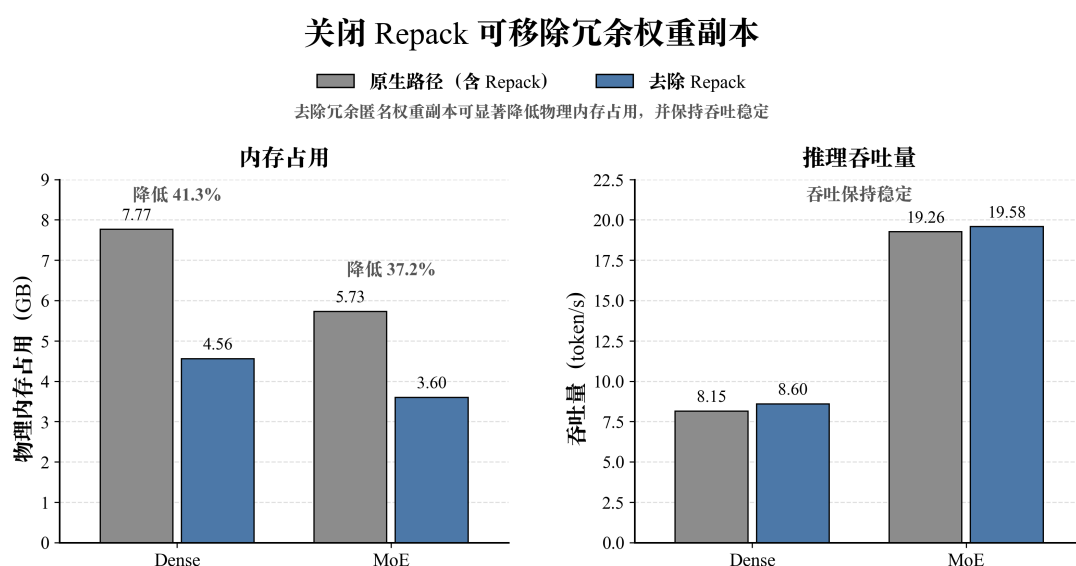


图 13: 关闭 Repack 前后的内存占用与推理吞吐

关闭 Repack 后, Dense 模型 RSS 由 7.77 GB 降至 4.56 GB, 降低 41.3%; MoE 模型由 5.73 GB 降至 3.60 GB, 降低 37.2%。与此同时, 两种模型的推理吞吐均未下降。

实验表明, 在 Flex-OS 已接管权重驻留后, Repack 产生的临时权重副本属于可消除的额外物理内存开销。去除该副本能够显著降低基础内存占用, 并为后续权重工作集管理释放更多可用空间。

5.2.2 Dense 模型权重驻留机制测试

Dense 模型具有逐层顺序执行特点，但每个生成 token 均需要访问完整模型权重。因此，在内存受限环境下，如果无法保持有效权重驻留集合，系统会退化为频繁磁盘加载模式。本系统针对 Dense 模型设计 Window + Flex Buffer 权重驻留机制，仅保持当前计算窗口附近层权重常驻，其余层根据访问顺序进行异步加载。

不同权重驻留策略下的主要实验结果如表 7 所示。

表 7: Dense Residency 机制效果

配置	Stream/token	Flex Wait	TPS(tok/s)
无显式 Residency	2684 MiB/token	25694 ms	1.77
当前 Residency 配置	612 MiB/token	6630 ms	5.18

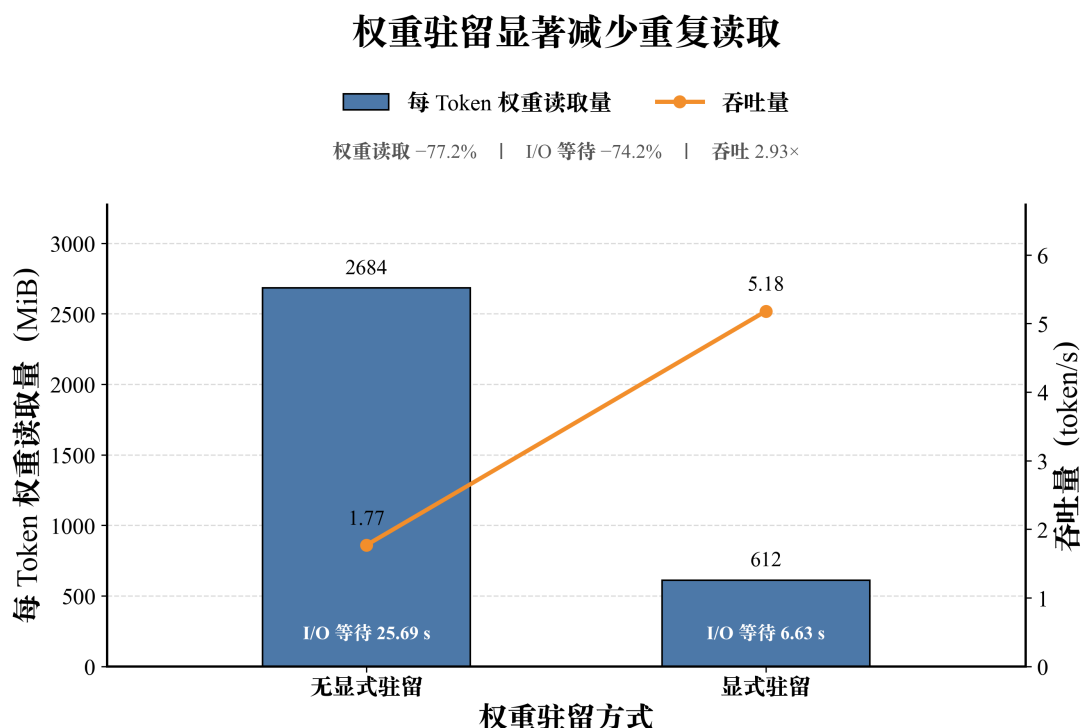


图 14: Dense 权重驻留机制效果

实验结果显示，引入显式权重 Residency 后，每 token 权重读取量由 2684 MiB 降低至 612 MiB，下降约 77.2%。同时，Flex Wait 从 25694ms 降低至 6630ms，吞吐由 1.77 tok/s 提升至 5.18 tok/s。

该结果说明，在 Dense 模型中，合理维护权重驻留集合能够显著减少重复 Weight Streaming，并降低同步 I/O 等待。

5.2.3 风险感知的权重锁定

增加权重驻留可以减少重复读盘，但同时会占用更多运行时内存。内存限制较紧时，单纯扩大驻留范围反而可能导致内存溢出。因此，本实验对比原自动策略、最大驻留策略和风险感知策略，验证不同权重锁定方式在低内存环境下的运行效果。

实验设置为 `memory.max=2000 MB`、上下文长度 2048 Token、输出 128 Token，每种配置重复测试 3 次。实验结果如表 8 和图 15 所示。

表 8: 风险感知权重锁定策略测试结果

策略	Stream/token	Flex Wait	TPS	运行结果
原自动策略	1682 MiB	37539 ms	2.31	稳定运行
最大驻留策略	—	—	—	3/3 OOM
风险感知策略	1382 MiB	34193 ms	2.53	稳定运行

风险感知锁定平衡驻留收益与 OOM 风险

内存上限 = 2000 MB | 上下文长度 = 2048 Token | 输出 128 Token | 重复 3 次

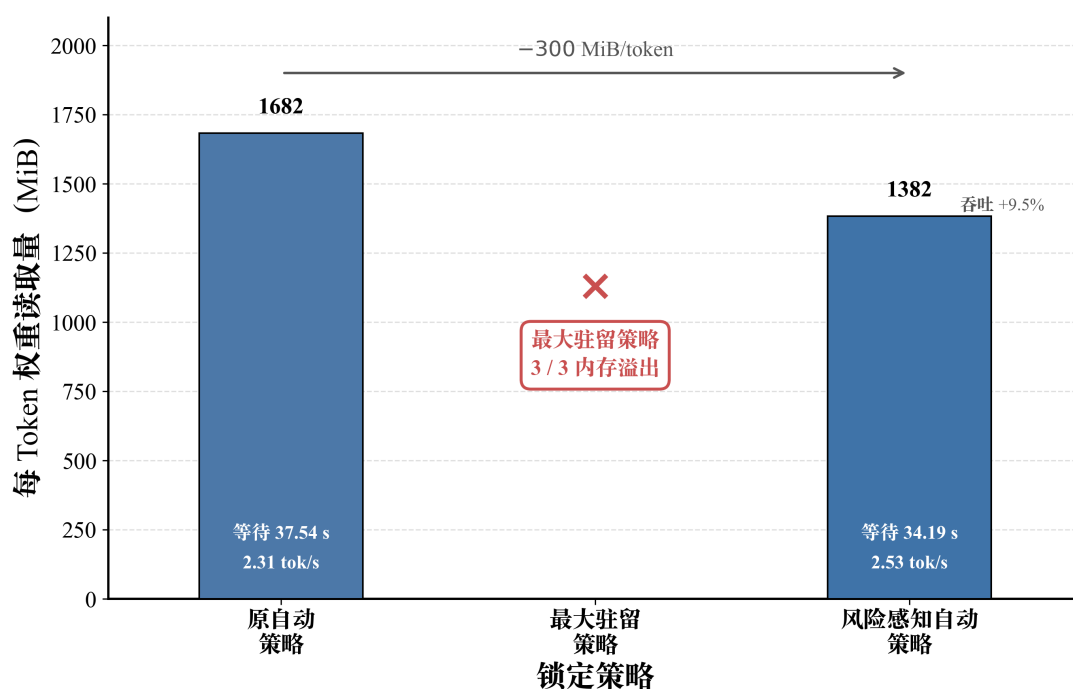


图 15: 不同权重锁定策略下的运行结果

结果表明，最大驻留策略在 3 次实验中均发生 OOM，说明权重并非驻留得越多越好。风险感知策略在保持稳定运行的同时，将 Stream/token 从 1682 MiB 降至 1382 MiB，减少约 300 MiB/Token，吞吐由 2.31 tok/s 提升至 2.53 tok/s。

该实验说明，在严格内存限制下，权重锁定需要在驻留收益与运行时内存安全之间进行权衡，而不能单纯追求最大化驻留。

5.2.4 不同权重驻留对象的影响

除驻留规模外，有限内存应优先保留哪些权重同样会影响 Dense 模型的 I/O 行为。为分析驻留对象选择的影响，本实验进一步对不同 Pin Policy 进行消融比较，包括无主动驻留、按 Tensor 大小、模型结构以及访问代价选择权重等策略。

实验设置为 `memory.max=3000 MB`、上下文长度 1024 Token、输出 64 Token，每种配置重复测试 3 次。实验结果如表 9 和图 16 所示。

表 9: 不同权重驻留对象选择策略的测试结果

策略	Flex Wait (ms)	TPS (tok/s)	说明
none	26448	1.80	无主动驻留
small-first	13386	2.88	优先驻留小尺寸 Tensor
attn-first	14141	2.85	优先驻留 Attention 权重
large-first	15969	2.62	优先驻留大尺寸 Tensor
cost-aware-balanced	17685	2.61	综合访问代价与权重大小
ffn-first	16851	2.57	优先驻留 FFN 权重
cost-aware	18403	2.54	按访问代价选择

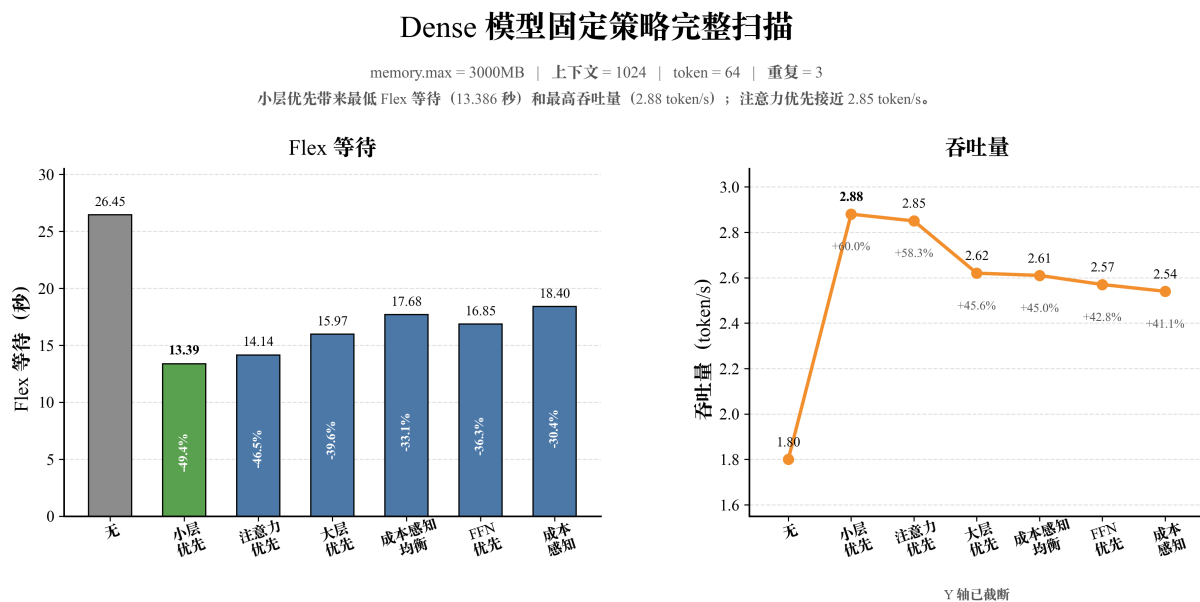


图 16: 不同权重驻留对象选择策略的性能对比

结果表明, 在相同内存预算下, 驻留对象的选择会明显影响 Dense 模型的 I/O 等待和推理吞吐。相比无主动驻留的基线, 所有主动策略均降低了 Flex Wait。其中 small-first 在本组实验中取得最高吞吐, TPS 由 1.80 tok/s 提升至 2.88 tok/s; attn-first 的结果与其接近。

同时, 不同策略之间仍存在明显差异, cost-aware 和 cost-aware-balanced 在本组测试条件下未取得最高性能。该实验说明, 权重驻留不仅需要确定驻留规模, 还需要考虑有限预算下的驻留对象选择; 本组结果用于说明不同选择策略对性能具有明显影响, 不代表某一固定策略在所有模型和负载下均为最优。

5.2.5 权重预取距离与自适应调节

在 Dense 模型中, 即使已有有效的权重驻留, 如果预取距离不足, 计算线程仍可能等待后续层权重加载完成。因此, 本系统进一步实现 Adaptive Ahead, 根据当前 I/O 状态和计算窗口动态调整预取距离, 并与固定 ahead0/1/2/4 配置进行对比。

实验设置为 memory.max=2400 MB、上下文长度 2048 Token、输出 128 Token, 每种配置重复测试 3 次。测试结果如表 10 和图 17 所示。

表 10: 不同权重预取距离与 Adaptive Ahead 策略对比

策略	Stream/token MiB/token	Flex (ms)	Wait	TPS (tok/s)	说明
ahead0	1682	36511		2.36	无前瞻预取
ahead1 fixed	1682	36404		2.38	预取距离较小
ahead2 fixed	1682	38501		2.33	预取距离较小
Adaptive Ahead	1036	23129		3.32	动态调整预取距离
ahead4 fixed	1682	377		4.94	固定较大预取距离

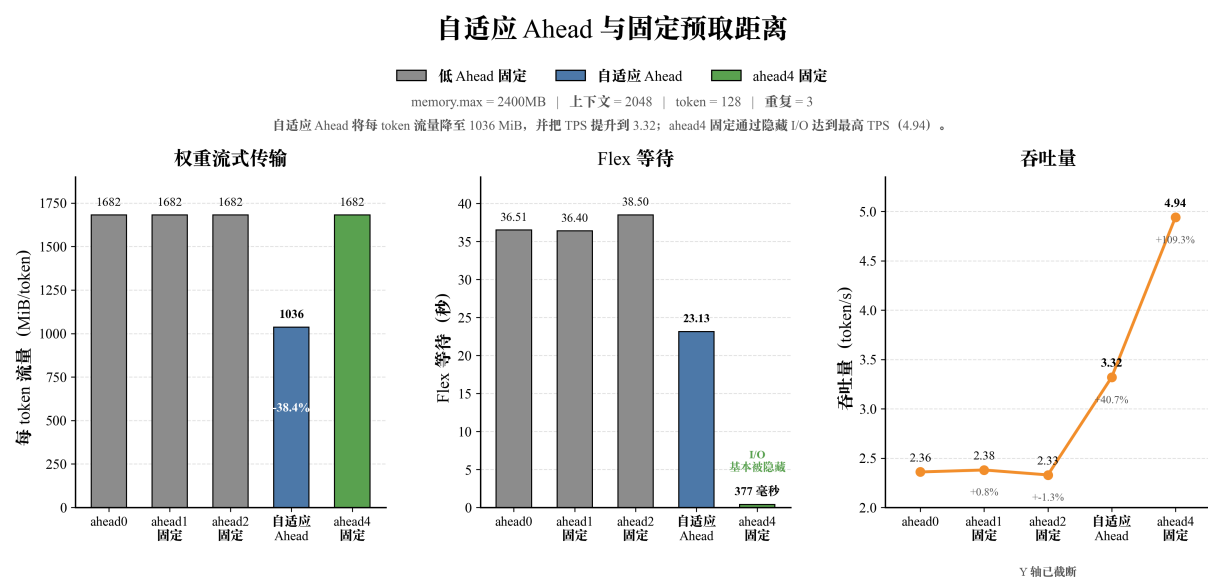


图 17: 不同权重预取距离下的性能对比

实验结果表明，较小的固定预取距离未能有效降低 I/O 等待；Adaptive Ahead 将 Stream/token 由 1682 MiB/token 降至 1036 MiB/token，Flex Wait 由 36511 ms 降至 23129 ms，TPS 提升至 3.32 tok/s。

固定 ahead4 在本组实验中取得最高吞吐 4.94 tok/s，说明 Dense 推理对预取距离较为敏感。Adaptive Ahead 的作用在于根据运行状态动态调整预取范围，而非保证在所有工作负载下超过固定最优参数。

5.3 MoE 模型权重管理测试结果

MoE 模型每个 Token 仅激活部分专家，其内存管理重点并非减少全部权重访问，而是在有限内存中维护主要活跃专家工作集，减少专家的反复驱逐与重新读取。本系统通过 MoE Buffer、专家缓存自动预算和压力感知调度，对专家权重的运行时驻留进行管理。

实验主要验证专家工作集边界、自动预算规划以及不同内存压力下的自适应调度效果，并结合运行时事件统计分析专家缓存抖动的产生原因。

5.3.1 MoE 专家工作集边界

首先测试不同 Expert Buffer budget 下专家缓存的运行状态。实验结果如表 11 和图 18 所示。

表 11: MoE 专家工作集边界测试

budget (MB)	Evictions	Read/token	TPS (tok/s)	状态
768	7392	598	1.97	严重抖动
1536	2710	272	2.23	驱逐明显
2048	618	132	6.22	接近工作集边界
2432	0	~0	11.57	主要工作集已覆盖
2560	0	~0	9.43	主要工作集已覆盖

MoE 专家缓存工作集边界

低于工作集：出现驱逐与读取放大 | ≥ 2432 MB：驱逐归零且每 Token 权重读取量接近 0

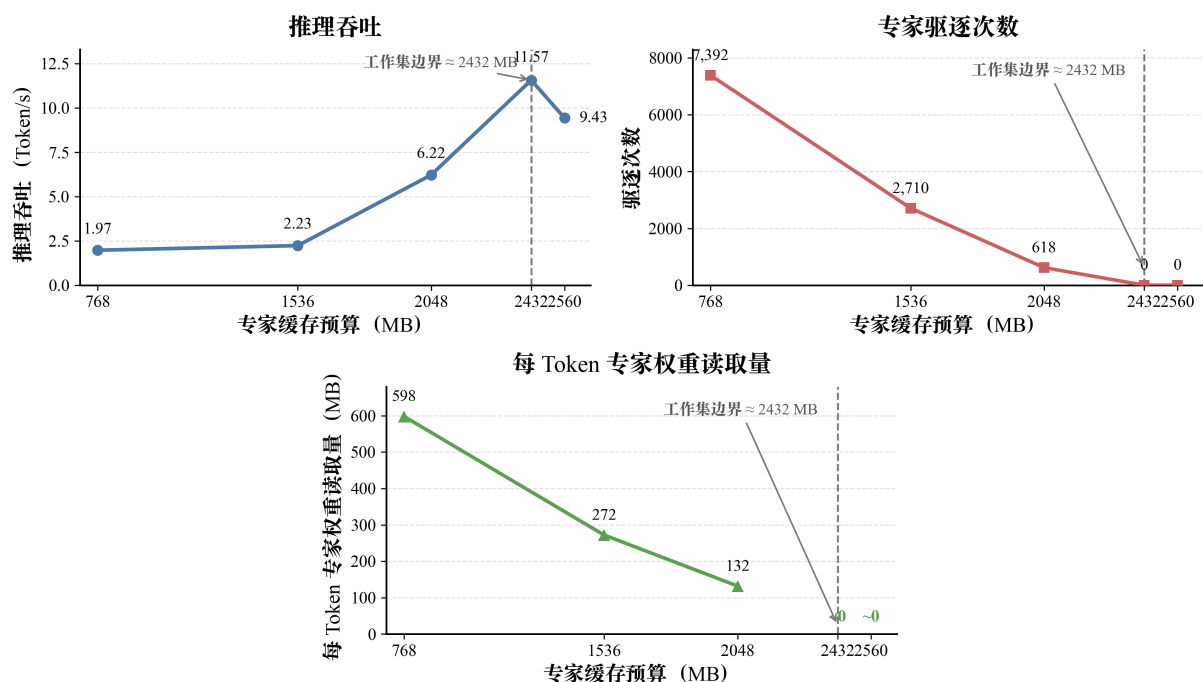


图 18: 不同专家缓存预算下的驱逐、权重读取与推理吞吐

结果表明，MoE Buffer 存在明显的专家工作集边界。当缓存预算较小时，活跃专家会被反复驱逐和重新读取，导致明显的 I/O 抖动。随着 budget 增大，Evictions 和 Read/token 持续下降；当预算增至约 2432 MB 时，Evictions 降至 0，Read/token 接近 0，说明缓存已经覆盖当前负载下的主要活跃专家工作集。

该实验说明，MoE 推理的关键并非缓存全部专家，而是在有限预算下尽可能覆盖主要活跃专家，减少重复驱逐和读盘。

5.3.2 MoE 专家缓存自动预算

固定的 Expert Buffer 大小难以同时适应专家工作集规模和严格的进程内存限制。因此，本系统根据运行时固定内存、活跃专家规模和安全余量自动计算可用的专家缓存预算。

在 `memory.max=1500 MB` 条件下，实验结果如表 12 所示。

表 12: MoE 专家缓存自动预算测试结果

策略	Expert Budget	OOM 次数	运行结果
固定 128 MB	128 MB	3/3	无法运行
固定 512 MB	512 MB	3/3	无法运行
固定 768 MB	768 MB	3/3	无法运行
固定 1024 MB	1024 MB	3/3	无法运行
自动预算	836 MB	0/3	稳定运行

自动规划找到可运行的专家缓存预算

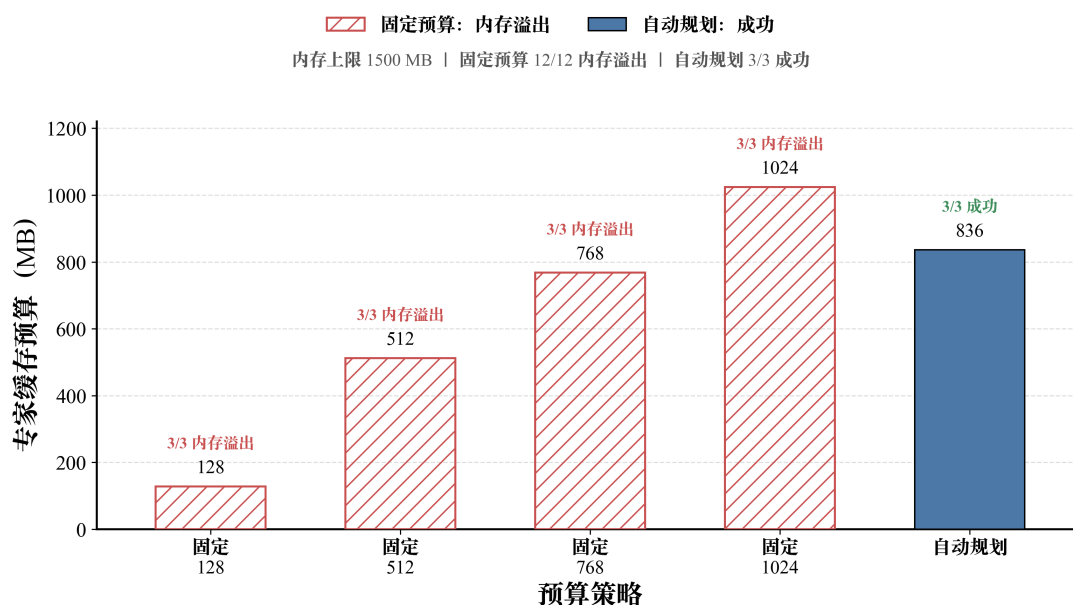


图 19: 固定专家缓存预算与自动预算的运行结果

实验结果显示, 在 1500 MB 严格内存限制下, 四组固定预算共 12 次测试均发生 OOM; 自动预算得到 836 MB 后, 3 次测试均稳定运行。

该结果说明, 专家缓存不能简单采用固定大小, 而需要结合整体运行时内存开销规划可运行的缓存预算。

5.3.3 压力感知专家调度

固定的专家保护范围难以适应不同内存压力: 保护范围过大可能挤占运行时内存, 保护范围过小则容易造成热点专家的反复驱逐。因此, 本系统根据实时内存压力动态调整专家保护范围。

实验结果如表 13 和图 20 所示。

表 13: 压力感知专家调度跨内存预算测试

memory cap	相对关闭自适应的 TPS 变化	状态	说明
1500 MB	+1.51%	提升	严格内存限制
1800 MB	+1.64%	提升	稳定收益
2000 MB	+0.99%	提升	中等内存限制
2400 MB	+0.51%	提升	较宽松内存限制

压力感知全自适应在不同内存预算下的表现

全自适应相对关闭自适应 | 4 个内存上限均为正收益 | 无 OOM 回退

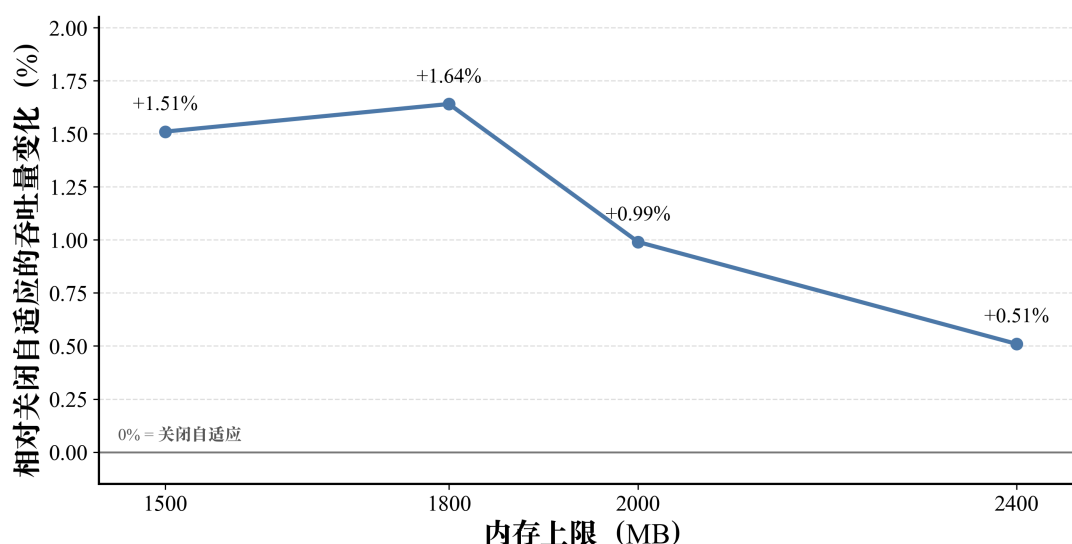


图 20: 压力感知专家调度在不同内存限制下的吞吐变化

结果显示，在 1500–2400 MB 四档内存限制下，压力感知专家调度的吞吐均高于关闭自适应策略，提升约 0.5%–1.6%。该机制的主要作用不是在单一内存条件下追求最高吞吐，而是在内存条件变化时动态调整专家保护范围，避免固定策略在不同压力下产生明显退化。

运行时行为分析 为进一步分析专家缓存发生性能退化的原因，本文通过 event-level trace 统计 Expert Buffer 运行过程中的 Eviction、Read Bytes 和 Cache Hit 等指标。测试条件为 `memory.max=2400 MB`、上下文长度 2048 Token、输出 256 Token，结果如表 14 和图 21 所示。

表 14: MoE 专家缓存运行时事件诊断

指标	Current Policy	Active Window Off	说明
Peak RSS	2399.9 MiB	1721 MiB	驻留压力差异明显
Evictions	16290	1962	专家驱逐明显增加
Read Bytes	18.44 GB	3.14 GB	出现 I/O 放大
Cache Hit	0.9539	0.9927	缓存命中率下降
TPS	7.07	8.60	推理吞吐下降

MoE 事件轨迹揭示当前策略下的抖动

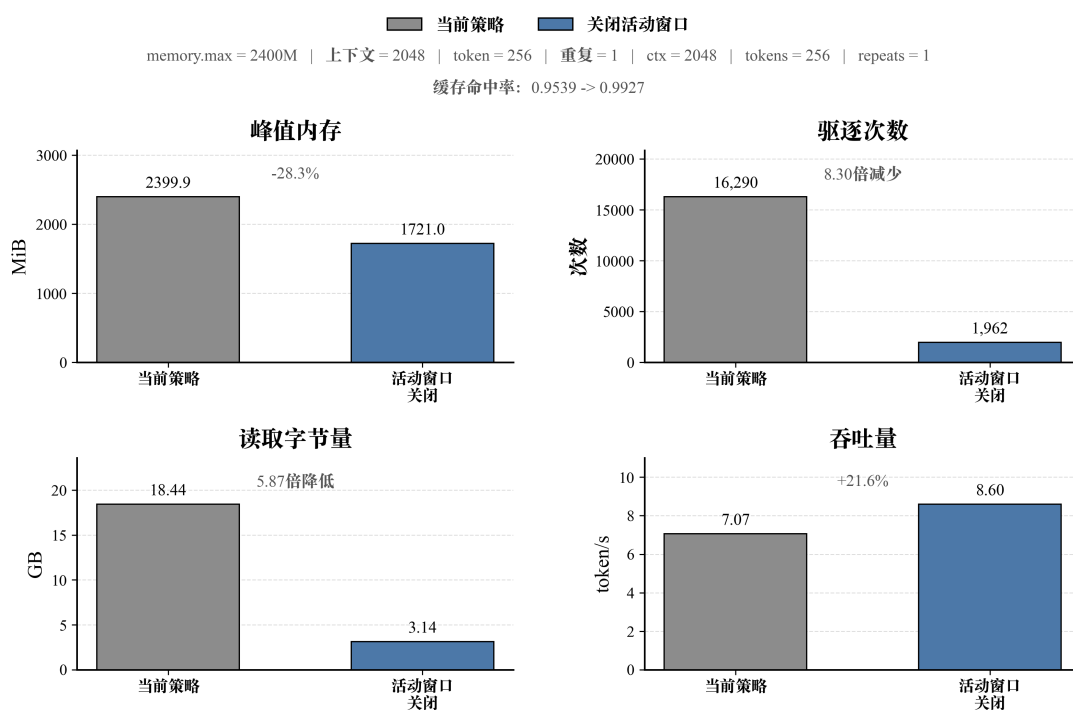


图 21: 不同专家保护策略下的运行时事件统计

事件统计表明, 过大的专家保护范围会提高驻留压力, 并进一步引起更多专家驱逐和重新读取。在该组实验中, Current Policy 相比 Active Window Off 的 Evictions 由 1962 次增至 16290 次, Read Bytes 由 3.14 GB 增至 18.44 GB, 同时 TPS 由 8.60 tok/s 降至 7.07 tok/s。

该结果进一步说明, 专家保护并非越多越好, 压力感知调度需要根据当前内存状态动态控制保护范围, 以减少缓存抖动和 I/O 放大。

5.4 KV Cache 管理模块测试结果

KV Cache 部分围绕真实物理驻留、会话恢复和多会话生命周期开展测试。容量侧使用 mincore 等物理驻留观测统计 KV tensor 的真实驻留页, 并分别记录 RELEASE 与 OFFLOAD 带来的物理内存收益; 恢复侧记录 backing read、prefault、scatter 和 restore wall time; 多会话侧通过 trace-driven replay 验证 Session 在 ACTIVE/IDLE/REVISIT/DEAD 等状态间的完整运行过程。

受控容量实验使用 Qwen1.5-MoE-A2.7B Q4_K_M 模型、CPU 推理和单 slot 长上下文 workload。容量控制实验采用 F32 K/V, 后续 KV 表示宽度和多会话实验采用 F16 K/V; F32/F16 仅在对应的受控 A/B 实验中直接比较。

5.4.1 KV 真实物理驻留控制

在固定模型、workload 与 binary 的 KV Resident Budget sweep 中, 以全驻留状态作为基线, 并逐步收紧 KV 物理驻留目标。实验覆盖 8 个预算点, 每个 case 重复 2 轮, 共 16 次运行。结果如表 15 和图 22 所示。

表 15: KV 物理驻留预算受控实验结果

策略 / target	实际 KV resident	相比全驻留节省	steady TPOT 变化
Resident	约 3.000 GiB	0	baseline
RELEASE 2.5 GiB	2.495 GiB	16.8%	+6.6%
RELEASE 2.0 GiB	1.995 GiB	33.5%	+6.0%
RELEASE 1.5 GiB	1.496 GiB	50.1%	+3.1%
RELEASE+ OFFLOAD 1.0 GiB	0.996 GiB	66.8%	+4.8%
RELEASE+ OFFLOAD 0.75 GiB	0.746 GiB	75.1%	+7.9%
RELEASE+ OFFLOAD 0.50 GiB	0.497 GiB	83.4%	+5.1%
RELEASE+ OFFLOAD 0.25 GiB	0.247 GiB	91.8%	+4.2%

目标预算可准确控制 KV 物理驻留

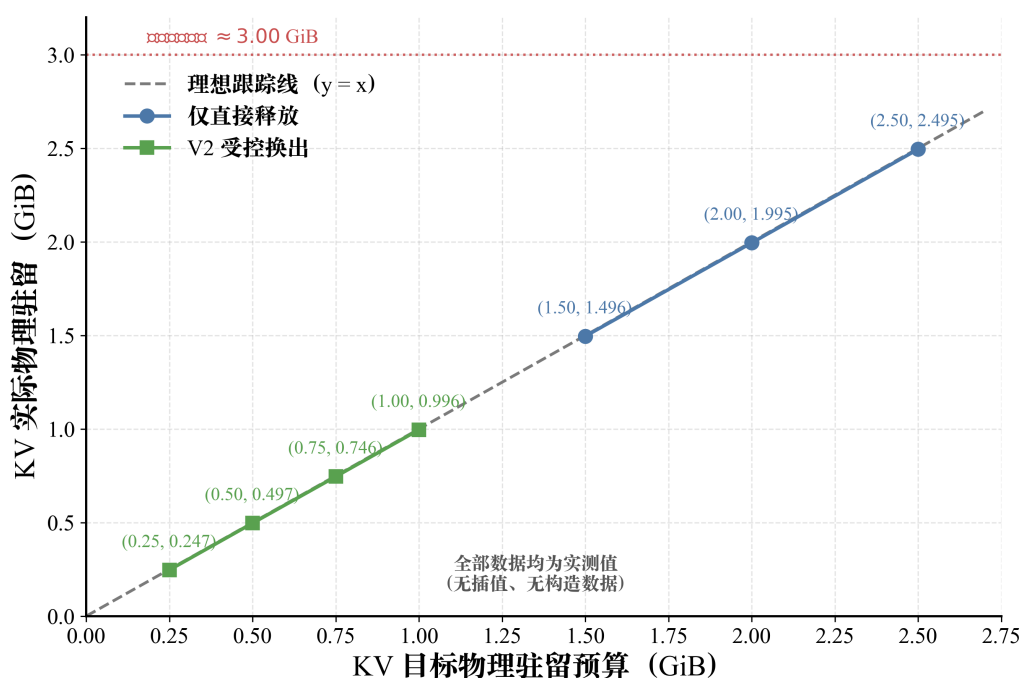


图 22: KV 目标物理驻留与实际物理驻留

随着 KV resident target 从 2.5 GiB 逐步收紧至 0.25 GiB，实际物理驻留由 2.495 GiB 连续下降至 0.247 GiB；相较约 3.00 GiB 的全驻留基线，最大减少 91.8%。各预算点的实际驻留均能够跟随目标变化，说明 KV Controller 可以按照真实物理驻留量控制 KV 工作集。

5.4.2 RELEASE 与 OFFLOAD 的物理收益归因

Flex-OS 根据 KV 生命周期采用两级回收策略：对于已经无需再次恢复的 KV，优先执行 RELEASE，直接释放物理页；当 RELEASE 后仍无法满足目标预算时，再对仍有会话价值但当前空闲的 KV 执行 OFFLOAD，写入后备存储后释放 DRAM。

实验结果如表 16 和图 23 所示。

表 16: RELEASE 与 OFFLOAD 的物理驻留收益

阶段	物理内存收益	实际 KV resident	说明
全驻留	—	约 3.00 GiB	Resident 基线
RELEASE	约 1.709 GiB	约 1.291 GiB	无需后备 I/O
RELEASE+OFFLOAD (0.25 GiB target)	OFFLOAD 额外 约 1.044 GiB	0.247 GiB	最大减少 91.8%

优先直接释放，必要时按需换出

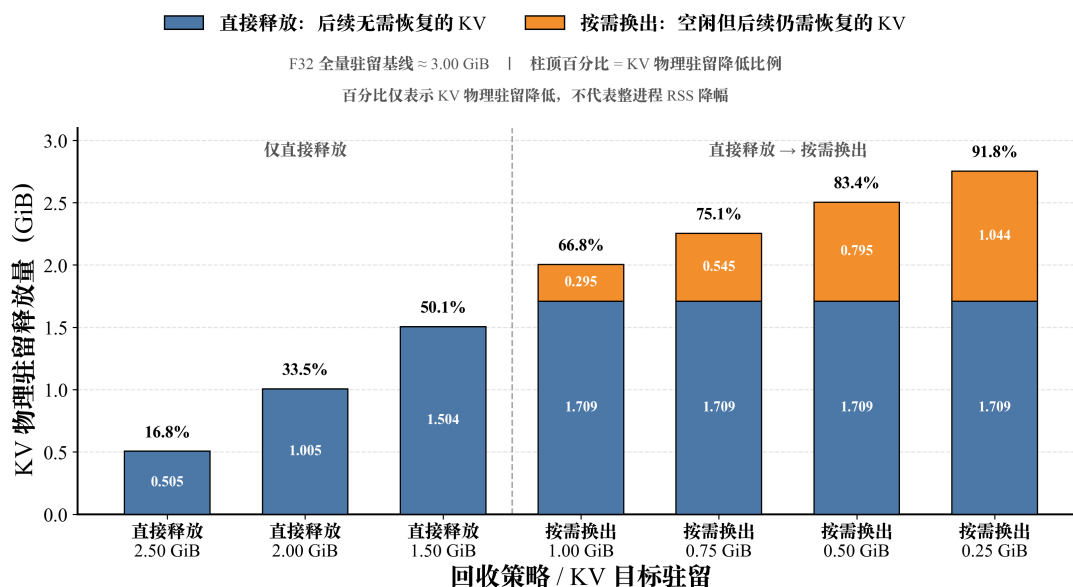


图 23: RELEASE 与 OFFLOAD 的物理内存收益

RELEASE 能够在不迁移有效 KV 的情况下直接回收无后续价值的数据。在最严格的 0.25 GiB 目标下，RELEASE 先释放约 1.709 GiB；仍有容量缺口时，OFFLOAD 再对空闲但需要保留的 KV 贡献约 1.044 GiB 的物理释放量。

该结果说明，优先 RELEASE、按需 OFFLOAD 可以避免对无需保留的数据进行无意义的后备存储写入，同时在更严格预算下继续压缩 KV 的物理驻留。

5.4.3 KV 物理驻留与会话恢复代价

进一步收紧 KV 物理驻留预算会释放更多 DRAM，但被 OFFLOAD 的有效 KV 也会增加，Session 重访时需要恢复更多数据。四个受控 OFFLOAD 预算点的结果如表 17 所示。

表 17: KV 物理驻留与会话恢复代价

实际 KV resident	恢复数据量	会话恢复等待
0.996 GiB	327 MiB	190.4 ms
0.746 GiB	576 MiB	277.2 ms
0.497 GiB	840 MiB	370.0 ms
0.247 GiB	1104 MiB	478.0 ms

KV 物理驻留与恢复代价曲线

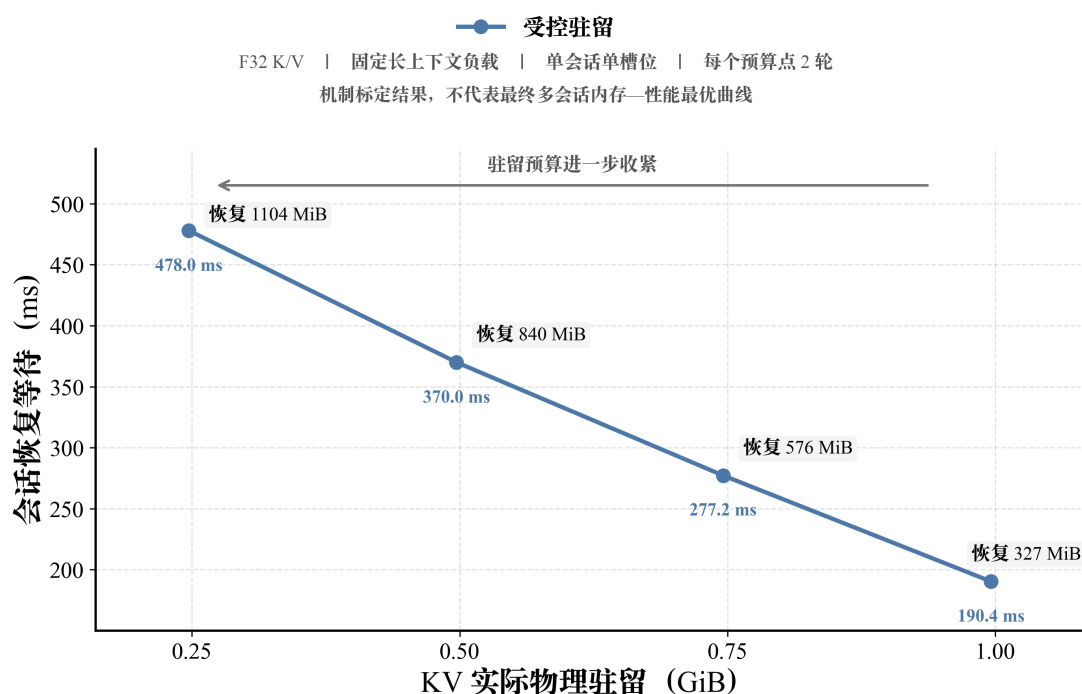


图 24: 不同 KV 物理驻留下的会话恢复代价

随着实际 KV resident 从 0.996 GiB 降至 0.247 GiB，恢复数据量由 327 MiB 增至 1104 MiB，会话恢复等待由 190.4 ms 增至 478.0 ms。说明 KV 物理驻留越低，Session 重访时需要承担越高的一次性恢复代价。

因此，KV 预算控制并非单纯追求最低驻留，而需要在物理内存占用与会话恢复延迟之间进行权衡。

5.4.4 KV 恢复路径优化

对于 OFFLOAD 到后备存储的 KV，Session 重访前必须完整恢复所需数据。为降低恢复关键路径开销，Flex-OS 分别从连续传输分组（Transfer Group）、读/恢复流水

(Read/Restore Pipeline) 和目标页预建立 (Destination Prefault) 三个方面优化 Exact Restore。

在 1024 Token workload 中, 16 个 SWAPPED block 被合并为 8 个 Transfer Group, 384 MiB backing payload 由逐 block 读取转换为连续 range read; 同一实验中, mincore 观测到约 381 MiB 的真实 KV resident drop。

在此基础上, Read/Restore Pipeline 按照

$$\text{READ}(G_{i+1}) \parallel \text{RESTORE}(G_i)$$

重叠下一组读取与当前组恢复。实验中后续 7 个 group 均未产生额外 exposed read wait 或 pipeline stall, staging 工作集峰值约 96 MiB。

Destination Prefault 在 tensor scatter 前预先建立目标物理页。三轮交错 OFF/ON A/B 均保持请求成功、输出一致和恢复门禁正常, 计时结果如表 18 所示。

表 18: KV 恢复路径 Prefault 三轮中位数

指标	Prefault OFF	Prefault ON	变化
Restore pipeline wall time	237,277 us	200,885 us	-15.34%
Restore scatter time	179,736 us	57,786 us	-67.85%
Prefault time	0	84,886 us	+84,886 us
Derived physical restore time	179,736 us	142,672 us	-20.62%
Scatter minor faults	97,536	0	全部前移
Prefault minor faults	0	97,536	全部前移
Major faults	0	0	不变

精确恢复路径：流水与预缺页消融

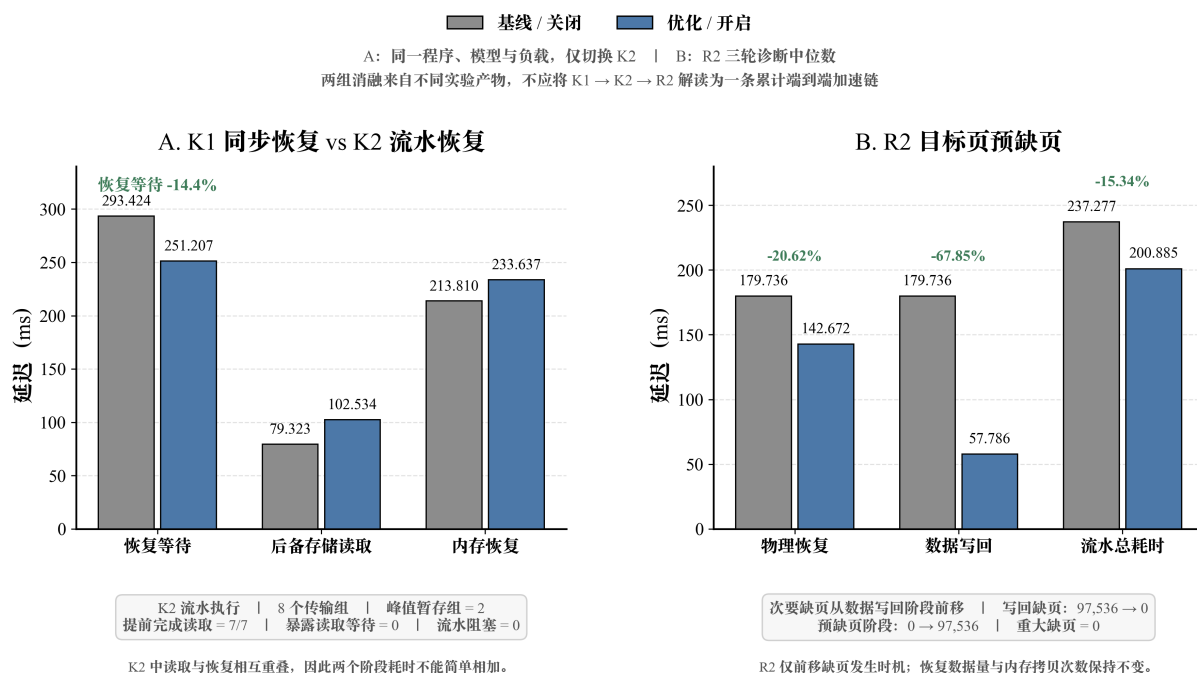


图 25: KV 恢复路径优化消融结果

Prefault 开启后, 97,536 个 minor page fault 从 scatter 阶段前移到 prefault 阶段, 使 scatter time 由 179.736 ms 降至 57.786 ms; 计入 Prefault 本身开销后, Derived physical restore time 由 179.736 ms 降至 142.672 ms, 降低 20.62%。

实验表明, KV 恢复开销不仅来自后备存储读取, 还包括目标物理页重新建立和数据写回; 连续读取、I/O 与恢复重叠以及目标页预建立能够共同降低恢复关键路径开销。

5.4.5 F16 KV 表示宽度与恢复数据量

KV 元素表示宽度直接影响物理驻留以及 OFFLOAD/Restore 的数据搬运量。该实验在近似相同的相对 resident 压力下比较 F32 与 F16 K/V, 用于分析 KV 表示宽度对驻留和恢复成本的影响, 与前述生命周期管理机制相互独立。

实验结果如表 19 和图 26 所示。

表 19: 近似等相对压力下 F32/F16 KV 对照

指标	F32	F16	变化
Full-KV physical resident	约 3.00 GiB	约 1.50 GiB	约减半
恢复数据量	840 MiB	465 MiB	-44.64%
Resume gate	471.318 ms	259.381 ms	-44.97%

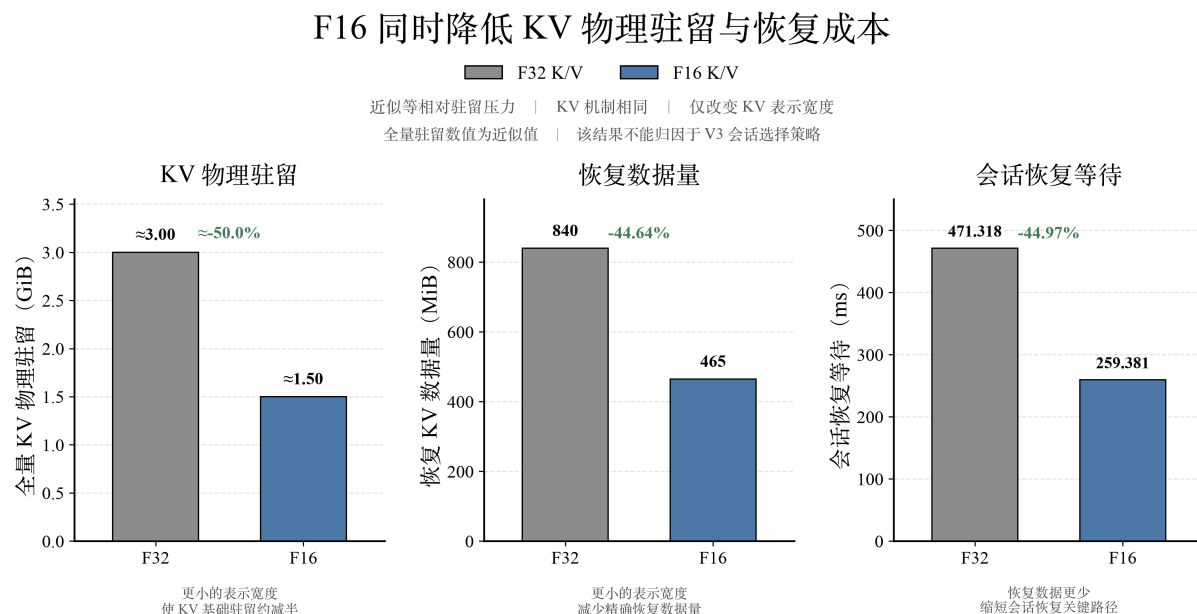


图 26: 近似等相对压力下 F32 与 F16 KV 对比

在相近的相对 resident 压力下，F16 将 Full-KV physical resident 从约 3.00 GiB 降至约 1.50 GiB；恢复数据量由 840 MiB 降至 465 MiB，Resume gate 由 471.318 ms 降至 259.381 ms。

该结果说明，降低 KV 表示宽度能够进一步减少物理驻留和恢复阶段的数据搬运量。该优化与 RELEASE、OFFLOAD 和 Exact Restore 生命周期控制属于相互独立的优化维度。

5.4.6 多会话生命周期验证

除受控单 Session 实验外，系统进一步将 Alibaba usage trace 映射到真实 llama-server，验证多会话条件下的 KV 生命周期执行过程。Trace 中的逻辑 Session 被稳定绑定到独立 slot，并按照真实请求到达和重访顺序执行。

运行结果如表 20 所示。

表 20: 多会话 KV 生命周期执行结果

验证内容	真实运行结果	作用
模型请求绑定	real tokenizer, direct_token_ids, effective n_ctx=8192	将 trace 请求绑定到实际模型 Token 与上下文配置。
多会话回放	2 个 logical session 稳定绑定不同 slot; HTTP 200; zero loss/duplicate; event order PASS	保持不同 Session 的独立生命周期 和请求顺序。
生命周期执行	3 个完成 turn; revisit_count=1、 ttl_expiry_count=2、 dead_count=2	验证 ACTIVE/IDLE/REVISIT/TTL/ DEAD 等状态转换。

结果表明, KV Controller 能够在真实多 Session 请求顺序下观察会话的活跃、空闲、重访和生命周期结束, 并驱动相应的 KV 状态转换。

5.5 全局自动调度测试结果

在前述 Dense 权重驻留、MoE 专家工作集和 KV Cache 管理模块独立测试的基础上, 本节进一步验证 Global Memory Governor 对不同内存工作集的统一协调能力。测试主要关注低内存条件下的可运行边界、代表性运行点的内存与吞吐, 以及不同模块之间的真实物理内存再分配。

容量边界实验采用固定 workload (ctx=2048, np=1, reqs=1, tokens=64), 在 7 个 cgroup memory.high 压力点下分别测试 Dense 与 MoE 模型。该组实验每个条件执行 1 次, 主要用于筛选最低可运行内存点, 不作为不同策略间的正式性能统计。

实验主要从以下三个方面展开:

1. 不同全局策略下的最低可运行内存门槛;
2. 成功运行时各策略的吞吐对比;
3. 内存占用与性能之间的权衡关系。

5.5.1 容量边界与可运行性

首先比较不同策略在逐步收紧 memory.high 时的最低成功运行点, 结果如表 21 所示。

表 21: 不同策略的最低可运行内存点

模型	策略	最低成功 memory.high
Dense	baseline	2300 MB
	kv_only	2300 MB
	weight_only	2000 MB
	combined_auto	2300 MB
MoE	baseline	2800 MB
	kv_only	2800 MB
	weight_only	1300 MB
	combined_auto	1300 MB

Dense 模型中,weight_only 将最低成功点由 2300 MB 下移至 2000 MB,而 combined_auto 在本组容量筛选中未进一步降低边界。说明在该 workload 下, Dense 低内存运行主要受权重工作集及其 I/O 行为影响,单独增加 KV Cache 回收未表现出明显的容量边界收益。

MoE 模型的变化更为明显: baseline 与 kv_only 均只能在 2800 MB 压力点成功运行,而 weight_only 与 combined_auto 的最低成功点均下降至 1300 MB,并在 7 个压力点中成功运行 6 个。该结果表明,在本组负载下,专家工作集管理是降低 MoE 最低可运行内存边界的主要因素。

5.5.2 代表性运行点的内存与吞吐

在容量边界之外,进一步选取统一内存配置下的代表性运行点,对比原生 llama.cpp 与 Flex-OS 自动联合模式的峰值 RSS 和推理吞吐。实验结果如表 22 所示。

表 22: Flex-OS 与原生 llama.cpp 代表性运行结果

模型	配置	峰值 RSS	吞吐 (tok/s)
Dense	原生 llama.cpp	2801.96 MB	3.090
	Flex-OS	2597.89 MB	4.727
MoE	原生 llama.cpp	2800.92 MB	7.339
	Flex-OS	1954.75 MB	11.463

Flex-OS 相比 llama.cpp 原生框架的吞吐与内存收益

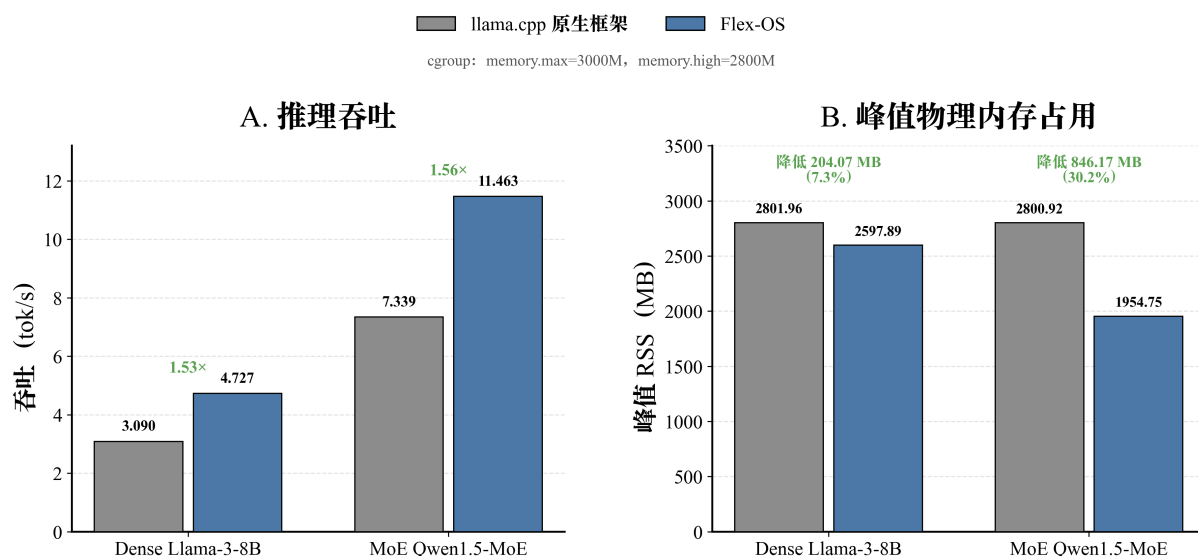


图 27: Flex-OS 与原生 llama.cpp 的内存占用和推理吞吐对比

在该代表性配置下，Dense 模型峰值 RSS 由 2801.96 MB 降至 2597.89 MB，吞吐由 3.090 tok/s 提升至 4.727 tok/s；MoE 模型峰值 RSS 由 2800.92 MB 降至 1954.75 MB，吞吐由 7.339 tok/s 提升至 11.463 tok/s。

该结果表明，在统一内存约束下，Flex-OS 能够通过不同工作集的协同管理降低物理内存占用，并保持较好的推理吞吐。

5.5.3 与 LiteRT-LM 的低内存可运行边界对比

为进一步比较 Flex-OS 在低内存环境下的可运行能力，本文选取 Google LiteRT-LM 作为外部框架对照。LiteRT-LM 测试采用 Qwen3-8B mixed_int4.litertlm 模型、CPU 后端、4 线程和磁盘缓存，并关闭 swap。Flex-OS 使用本项目 Dense 与 MoE 测试模型。由于两组实验的模型、量化格式和运行栈不同，本实验主要比较低内存可运行边界及受限配置下的运行表现，不进行同模型绝对性能归因。

实验结果如图 28 所示。

Flex-OS 与 LiteRT-LM 的低内存可运行边界与吞吐对比

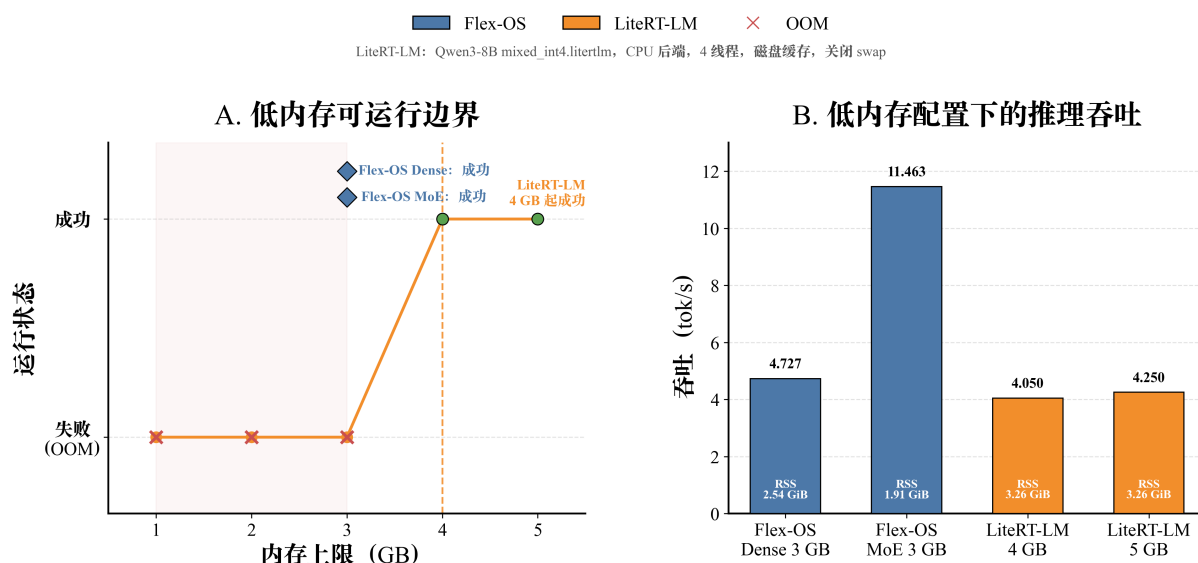


图 28: Flex-OS 与 LiteRT-LM 的低内存可运行边界与吞吐对比

LiteRT-LM 在 1–3 GB 内存限制下均发生 OOM，从 4 GB 开始成功运行；Flex-OS 的 Dense 与 MoE 测试模型均能够在 3 GB 内存限制下完成推理。其中，Flex-OS Dense 的峰值 RSS 约为 2.54 GiB，吞吐为 4.727 tok/s；MoE 的峰值 RSS 约为 1.91 GiB，吞吐为 11.463 tok/s。

该结果表明，在本项目测试模型和运行环境下，Flex-OS 能够支持较低内存限制下的模型推理。需要注意的是，LiteRT-LM 当前对照路径与 Flex-OS 使用的模型和运行栈并不相同，且不包含本项目的 MoE 专家管理路径，因此图中的吞吐数据仅用于展示各自在受限内存配置下的运行状态。

5.5.4 跨模块物理内存再分配

为验证 Global Memory Governor 是否能够将一个工作集释放的真实物理内存重新分配给其他模块，系统进一步构造 KV Cache 与 MoE Buffer 同时启用的受控实验。只有在 KV Controller 确认实际物理驻留下降后，对应内存额度才能被 Global Memory Governor 重新分配。

实验结果如表 23 所示。

表 23: KV 到 MoE 跨模块物理内存再分配结果

阶段	实验结果	说明
KV OFFLOAD	8 Blocks, 物理释放 99,090,432 B	确认 KV 真实物理驻留下降
物理收益确认	99,090,432 B	Global Memory Governor 确认可用物理收益
可再分配额度	67,108,864 B (64 MiB)	实际分配给 MoE Buffer
MoE 专家预算	1,207,959,552B → 1,275,068,416 B	专家缓存预算增加 64 MiB
KV Exact Restore	恢复 8 Blocks, 读取 100,663,296 B	恢复完成后继续计算
输出一致性	16/16 Token 完全一致	与相同 MoE 预算的 Resident Control 一致

实验中, KV Controller 对当前空闲但仍需保留的 KV 执行 OFFLOAD, 8 个 Block 产生 99,090,432 B 的真实物理内存释放。Global Memory Governor 确认该物理收益后, 将其中 67,108,864 B 的可用额度分配给 MoE Buffer, 使专家缓存预算增加 64 MiB。

当原 Session 再次访问时, 系统完成 8 个 KV Block 的 Exact Restore 后继续计算, 最终输出与相同 MoE 预算、KV 保持驻留的对照路径 16/16 Token 完全一致。

该实验验证了 KV 物理内存释放、全局额度确认、MoE 预算增加以及 KV 恢复之间的跨模块资源再分配闭环。该实验用于验证物理内存协调与计算正确性, 不作为吞吐或恢复延迟的性能提升结论。

5.6 系统测试结果总结

综合上述实验结果, Flex-OS 能够针对 Dense 权重、MoE 专家和 KV Cache 不同的访问特征进行运行时内存管理, 并通过 Global Memory Governor 在统一内存约束下协调各类工作集。

主要实验结论如下:

1. **Dense Flex** 能够减少重复权重读取, 并在低内存条件下控制驻留风险。

通过消除冗余权重副本和维护有限层工作集, Dense 模型的每 Token 权重读取量由 2684 MiB 降至 612 MiB, I/O 等待明显下降; 风险感知的权重锁定能够避免单纯扩大驻留范围导致的 OOM。

2. MoE Buffer 的关键在于覆盖主要活跃专家工作集并动态调整缓存预算。

随着专家缓存覆盖主要工作集，Evictions 和重复读取显著下降；自动预算和压力感知调度能够根据内存约束调整专家驻留规模，减少低内存条件下的专家驱逐与重加载。

3. KV Controller 能够直接控制 KV Cache 的真实物理驻留，并在内存收益与恢复代价之间进行权衡。

受控实验中，KV 真实物理驻留可由约 3.00 GiB 连续降低至 0.247 GiB，最大减少 91.8%。随着驻留进一步降低，Session 重访时的恢复数据量和恢复等待同步增加，说明 KV 管理需要同时考虑物理内存收益与会话恢复成本。

4. Global Memory Governor 能够协调不同工作集的资源使用，并形成跨模块物理内存再分配闭环。

全局实验表明，Flex-OS 能够在降低峰值 RSS 的同时维持较好的推理吞吐；进一步的 KV-MoE 协同实验验证了“KV 真实释放 → 全局确认 → MoE 预算增加 → KV 精确恢复”的资源再分配过程，并保持模型输出一致。

总体来看，Flex-OS 并非采用单一缓存策略降低内存，而是根据 Dense 权重、MoE 专家和 KV Cache 不同的访问规律分别维护对应工作集，并由 Global Memory Governor 根据运行时内存压力统一协调回收、驻留和资源再分配，从而提高受限内存环境下的大模型推理可运行性和资源利用效率。

6 当前系统不足与未来方向

尽管本系统已经完成核心功能验证，但当前仍存在若干限制和后续优化空间。

6.1 运行时资源决策的自适应能力仍需进一步提升

当前系统已经具备基于运行时状态进行动态资源调整的能力，并能够根据内存压力、数据访问特征和实际执行反馈持续修正资源分配策略。但现阶段实验主要集中于有限的模型类型、上下文规模和并发负载组合，对于突发请求、长时间多会话运行以及不同访问模式频繁切换等复杂场景，系统调度策略的稳定性和长期收敛性仍缺乏充分验证。未来可进一步扩大动态负载和长期运行测试范围，重点分析频繁资源调整是否会引入额外抖动、策略切换开销或局部性能波动，并进一步提高系统在复杂运行环境下的稳定性和鲁棒性。

6.2 极端内存约束下仍存在内存占用与推理性能的权衡

实验结果表明，通过主动控制数据驻留和按需加载能够显著降低大语言模型推理所需的物理内存，并使部分原本无法运行的低内存配置具备推理能力。但当可用内存持续减小时，更多模型数据和运行时状态需要在内存与外部存储之间频繁迁移，系统瓶颈会逐渐由内存容量转化为存储 I/O，进而增加推理等待时间。因此，未来优化不应单纯追求最低物理内存占用，而应进一步研究内存占用、数据迁移开销与推理性能之间的动态平衡，根据实际资源条件选择更合适的运行点。

6.3 系统泛化能力与实验验证范围仍有进一步扩展空间

当前系统已经在典型 Dense 模型、MoE 模型、长上下文以及多会话场景下完成验证，但实验仍主要集中于有限模型规模、CPU 推理路径和当前存储环境。不同模型结构、量化格式、上下文长度及硬件平台可能具有不同的访存特征和最优资源配置。未来可进一步扩展至更大规模模型、更多端侧设备和不同存储带宽环境，并探索 CPU、GPU、NPU 等异构设备下的统一运行时内存管理，从而验证并提升系统在不同模型和硬件条件下的通用性与可扩展性。

7 大语言模型使用说明

项目开发过程中，我们合理使用了大语言模型作为辅助工具，具体使用方式如下：

1. 文献检索与资料整理：借助开源大语言模型（如 DeepSeek V4 等）对相关研究论文进行语义检索与摘要生成，辅助快速定位技术路线和关键参考文献，提高调研效率。
2. 开源框架梳理与分析：利用 LLM 对 llama.cpp、vLLM、SGLang 等开源推理框架的代码仓库结构、文档说明和实现机制进行快速梳理与对比分析，辅助理解各框架的权重管理、KV Cache 组织方式以及相关优化技术，为系统设计与方案选型提供参考。
3. 代码开发辅助：在系统实现过程中，使用 LLM 辅助生成代码框架、编写测试脚本、调试错误及优化代码结构，所有生成代码均经人工审阅、修改和验证，确保符合项目需求和正确性。

需要特别说明的是，所有技术方案设计、核心算法实现、实验设计、数据分析与结论均完全由本项目成员独立完成，LLM 仅作为辅助工具使用，项目最终成果的完整性与真实性由本项目成员全权负责。

参考文献

- [1] Zixuan Zhou, Xuefei Ning, Ke Hong, et al., A Survey on Efficient Inference for Large Language Models, arXiv preprint, 2024.
- [2] Hongchao Du, Shangyu Wu, Arina Kharlamova, et al., FlexInfer: Breaking Memory Constraint via Flexible and Efficient Offloading for On-Device LLM Inference, arXiv preprint, 2024.
- [3] Keivan Alizadeh, Iman Mirzadeh, Dmitry Belenko, et al., LLM in a flash: Efficient Large Language Model Inference with Limited Memory, arXiv preprint, 2024.
- [4] Zhenliang Xue, Yixin Song, Zeyu Mi, et al., PowerInfer-2: Fast Large Language Model Inference on a Smartphone, arXiv preprint, 2024.
- [5] Zhiyuan Fang, Zicong Hong, Yuegui Huang, et al., Fate: Fast Edge Inference of Mixture-of-Experts Models via Cross-Layer Gate, arXiv preprint, 2024.
- [6] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, et al., Efficient Memory Management for Large Language Model Serving with PagedAttention, in *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023, pp. 611–626.
- [7] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, et al., SGLang: Efficient Execution of Structured Language Model Programs, arXiv preprint, 2024.
- [8] Yuhang Li, Rong Gu, Chengying Huan, et al., HotPrefix: Hotness-Aware KV Cache Scheduling for Efficient Prefix Sharing in LLM Inference Systems, arXiv preprint, 2025.
- [9] Krishna Teja Chitty-Venkata, Jie Ye, Xian-He Sun, et al., PagedEviction: Structured Block-wise KV Cache Pruning for Efficient Large Language Model Inference, arXiv preprint, 2024.
- [10] Ramya Prabhu, Ajay Nayak, Jayashree Mohan, et al., vAttention: Dynamic Memory Management for Serving LLMs without PagedAttention, in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2025.